

不确定环境下山区乡村农作物种植策略优化

——基于随机规划与 CVaR 风险度量

摘要：针对某山区乡村 54 个地块、41 种作物在 2024—2030 年的种植规划问题，本文在地类适种性、水浇地种植制度、轮作及豆类种植要求等约束下，构建由确定性规划、随机规划和相关风险分析组成的递进模型。

首先，根据 2023 年种植记录重构作物一季次层面的销量基准，建立混合整数规划模型，分别求解超产滞销和超产按半价销售两种情形。两种方案的七年名义利润分别为 4104.45 万元和 6390.94 万元，超产量则由 16.11 万斤增至 949.19 万斤。结果表明，超产产品的处置方式不仅影响销售收入，也会改变模型对高产作物的种植激励，因此第二市场的实际消化能力是影响方案选择的重要条件。

其次，根据题目给出的销量、亩产、成本和价格变化范围，采用拉丁超立方抽样生成 60 个情景，并利用样本平均近似建立随机规划模型，以期望利润和 90% 下尾 CVaR 共同衡量收益与风险。所得 Q2 方案的样本内期望利润和 CVaR 分别为 6964.06 万元和 6831.29 万元，与相应单目标有效上界的相对差距分别为 0.0084% 和 0.0744%。进一步在 3000 条独立样本外路径上评价固定方案，Q2 方案的平均利润和 CVaR 分别为 6963.37 万元和 6820.53 万元，均高于问题一中的超产半价方案，说明其在当前不确定性设定下具有更好的收益与下尾风险表现。

在此基础上，问题三将 Q2 扩展为相关经营环境：引入天气、作物组市场和投入成本共同因子，以稀疏价格弹性描述替代与互补关系，并把部分价格、成本增长率改为题面区间内的随机变量。在统一的 3000 条 Q3 相关样本路径上，Q3 候选方案的平均利润和 CVaR 均略低于 Q2 方案；两方案各作物累计种植面积的相关系数为 0.999608。该结果表明，在当前 Q3 参数组下，重新优化没有带来可验证的样本外优势，主要作物结构也没有发生实质改变；共同冲击强度的敏感性结果进一步显示，依赖增强主要压低下尾 CVaR。

综合农业约束核验、优化结果、独立样本外评价和敏感性分析，本文推荐 Q2 方案作为 2024—2030 年的基准种植方案，并利用 Q3 模型评估共同冲击下的尾部风险。实际实施前，应重点核实超产产品的市场吸收能力，并根据后续获得的销量、价格、天气和投入成本数据及时更新风险参数。

关键词：种植规划；混合整数规划；样本平均近似；CVaR；共同因子；样本外评价

1 问题重述

某山区乡村共有平旱地、梯田、山坡地、水浇地、普通大棚和智慧大棚六类耕地，共 54 个地块、1213 亩。题目给出了 2023 年的实际种植记录，以及不同作物在各类土地和季次下的亩产量、种植成本与销售价格等数据，要求制定 2024—2030 年的农作物种植方案 [1]。

种植安排除受土地面积限制外，还需满足不同地类的适种要求、季次安排和轮作条件。水浇地需要在单季水稻和两季蔬菜之间选择年度种植方式；同一地块不能连续重茬；同时，每块土地还需在连续三年内安排豆类作物。除此之外，题目还要求兼顾种植区域不宜过于分散、单个地块种植面积不宜过小等实际管理要求。

问题一假设未来各项经营参数按题目给定的基准保持稳定，分别考虑超出预期销售量的产品无法销售，以及超出部分按正常价格的 50% 销售两种情形，求 2024—2030 年的最优种植方案。

问题二进一步考虑未来市场和生产条件的不确定性。其中，小麦和玉米的预期销售量具有增长趋势，其他作物的销售量存在波动，亩产量、种植成本以及部分作物销售价格也随年份发生变

化。需要在这些不确定因素下给出兼顾收益与风险的种植方案。

问题三在问题二的基础上进一步考虑作物之间可能存在的替代性、互补性，以及预期销售量、销售价格和种植成本之间的相关关系，并据此给出新的种植方案，同时与问题二的结果进行比较。

2 问题分析

2.1 总体分析

本题属于多年度、多地块、多作物的农业种植规划问题。决策变量不仅包含各作物的种植面积，还同时受到地类适种性、季次制度、连续重茬限制和三年豆类轮作要求的约束。因此，不能将全部耕地简单合并为一个总面积进行分配，而应保留地块、年份和季次三个层面的种植结构。

其中，报表中的“第一季”“第二季”主要用于产量和收益核算，而轮作约束需要按照真实的种植先后关系判断。对于一年两季的地块，同一年第一季与第二季、上一年第二季与下一年第一季都可能构成相邻种植机会；2023 年的实际种植情况也会直接影响 2024 年的首个种植安排。由此，本文在统一的农业可行域上逐步增加经济与随机因素，使三个问题保持相同的基本决策结构，并能够直接比较不同假设对方案的影响。

2.2 问题一分析

问题一中的亩产量、种植成本、销售价格和预期销售量均按确定值处理，因此可将其建立为确定性混合整数规划模型。连续变量表示各地块在不同年份、季次上的作物种植面积，0-1 变量用于表示种植单元是否启用以及水浇地年度种植制度。

经济核算的关键在于区分正常销售量与超产量。正常销售量受到实际产量和预期销售量的共同限制；对于超过预期销售量的部分，分别按照“无法销售”和“按正常价格 50% 销售”两种规则计算收入。两种情形采用完全相同的农业约束，只改变超产部分的收益规则，因此可以直接比较超产处置方式对利润和种植结构的影响。

2.3 问题二分析

问题二中，预期销售量、亩产量、销售价格和种植成本均可能随未来年份变化。若只使用这些随机参数的均值进行优化，会忽略不利年份下的收益损失；仅最大化期望利润，也可能得到平均收益较高但下尾风险较大的方案。因此，需要同时刻画方案的平均收益和不利情景下的利润表现。

本文根据题目给出的变化范围生成有限个未来情景，并采用样本平均近似方法将随机优化问题转化为可求解的有限情景模型。在所有情景中，种植面积和离散种植状态保持一致，即种植方案在未来情景实现之前确定；各情景只改变亩产量、需求、价格和成本，并据此形成不同的利润结果。风险指标采用利润的下尾 CVaR，用于衡量较差情景中的平均利润水平，从而在期望收益与下尾风险之间进行比较。

2.4 问题三分析

问题二主要描述各随机因素自身的波动，但实际农业生产中，多种作物可能同时受到天气、市场环境和投入品价格变化的影响，因此不同随机变量之间可能存在共同波动。题目同时提出

作物之间的替代性与互补性，需要进一步刻画价格变化对需求关系的影响。

由于题目附件只提供单个基准年份的数据，无法直接估计 41 种作物之间完整的经验相关矩阵。为避免引入大量缺乏依据的两两相关系数，本文采用少量共同因子描述天气、作物组市场和投入成本带来的相关冲击，并用稀疏价格弹性关系表示部分作物之间的替代和互补作用。Q3 保持问题二的农业可行域和种植决策结构不变，但除改变依赖结构外，还把 Q2 中固定的部分价格、成本年增长率改为题面区间内的随机变量。因此，后文把 Q2、Q3 置于同一组 Q3 相关路径上作公平比较，并把差异解释为整个“Q3 相关经营环境”的综合作用，而不把它全部归因于相关系数。

图 1 概括了三个模型和验证环节之间的关系。每一步都保留前一步已经核验的农业可行域；最终建议同时接受农业约束复查、单目标上界证书和独立样本外评价。



图 1: Q1-Q3 模型演进与验证架构。Q3 不仅加入共同因子和稀疏弹性，还按题面区间随机化部分价格、成本增长率。

3 模型假设

题目已经明确给出的地类适种性、水浇地种植制度、轮作要求和豆类种植要求均作为模型的农业约束直接处理，不再重复列为假设。对于题面未给出但建模必须明确的部分，作如下假设。

- 销量基准假设。**附件未给出各作物在 2023 年的预期销量，本文以 2023 年实际种植记录与亩产参数计算得到的理论产量，作为相应“作物-季次”层面的基准预期销量。该量仅作为销量代理，不视为真实市场销量。
- 基准价格假设。**对附件中以区间形式给出的销售单价，取区间中点作为 2023 年的代表性价格。该价格用于建立确定性基准及后续价格变化路径，不解释为实际成交均价。
- 利润口径假设。**规划期目标采用 2024-2030 年的名义种植利润，仅计入销售收入和附件给出的种植成本；由于题面未提供运输、仓储、税费、融资、固定资产折旧等数据，模型不额外引入上述成本，也不进行跨期折现。
- 土地利用假设。**各地块在满足题面农业约束的前提下允许部分休耕，即各季种植面积之和不超过地块容量，但不要求必须满种。题目只定性提出种植不宜过于分散、单块种植面积不宜过小，未给出数值阈值，因此主模型不人为设置最小种植面积或最大地块数约束，相关问题在结果分析中另行评价。
- 重茬处理假设。**题目指出连续重茬会导致减产，但未给出具体减产比例。为避免补造减产

参数，本文采用保守处理：同一地块的相邻物理种植机会不安排同一种作物；2023 年最后一次实际种植状态同时作为 2024 年首个种植机会的边界条件。

6. **豆类轮作近似。**题目要求每块土地在任意连续三年内至少种植一次豆类，但未给出“种植一次”所对应的最小面积。本文以滚动三年内豆类累计种植面积不低于该地块面积作为线性近似，并在首个 2023–2025 窗口中计入 2023 年的实际豆类种植面积。
7. **问题二的概率设定。**题目只给出未来销量、亩产、成本和价格的变化范围，没有给出概率分布。本文在问题二中将相应随机扰动设为给定区间上的均匀分布，并在不同随机创新之间采用独立设定；CVaR 置信水平取 90%。因此，问题二的风险结果均应理解为条件于上述概率设定的情景评价。
8. **问题三的相关结构设定。**问题三采用共同因子描述天气、市场和投入成本冲击，并用稀疏价格弹性描述部分作物间的替代与互补关系。共同因子载荷和弹性点值均为模型参数设定，外部文献只用于支持影响方向和数量级，不能视为该村历史数据的统计估计值；其影响将在敏感性分析中检验。

4 符号说明

为避免报表中的“第一季、第二季”与轮作所需的真实种植先后关系混淆，本文同时使用报表季次和物理种植机会两套索引。报表季次用于产量、销售和利润核算，物理种植机会仅用于描述相邻种植关系。主要符号如表 1 所示，其余局部符号在首次出现时定义。

表 1: 主要符号说明

符号	含义	单位
$i \in \mathcal{I}$	地块索引	—
$j \in \mathcal{J}$	作物索引	—
$t \in \mathcal{T}$	年份索引, $\mathcal{T} = \{2024, \dots, 2030\}$	—
$s \in \mathcal{S}$	报表季次索引	—
$\omega \in \Omega$	随机场景索引	—
$h \in \mathcal{H}_i$	地块 i 上的物理种植机会索引	—
$\mathcal{E}_i$	地块 i 上相邻物理种植机会构成的边集	—
A_i	地块 i 的可用面积	亩
ℓ_i	地块 i 的地类	—
$\mathcal{F}$	通过适种性审查的“地块–作物–年份–季次”组合集	—
x_{ijts}	地块 i 在年份 t 、季次 s 种植作物 j 的面积	亩
z_{ijts}	对应种植单元是否启用的 0–1 变量	—
r_{it}	水浇地 i 在年份 t 的年度种植制度变量	—
$y_{j\ell s}$	作物 j 在地类 ℓ 、季次 s 下的基准亩产	斤/亩
$c_{j\ell s}$	作物 j 在地类 ℓ 、季次 s 下的基准亩成本	元/亩
$p_{j\ell s}$	作物 j 在地类 ℓ 、季次 s 下的基准销售单价	元/斤

符号	含义	单位
D_{js}	作物 j 在季次 s 的基准预期销量代理	斤
$q_{j\ell ts}^\omega$	场景 ω 下正常市场销售量	斤
$e_{j\ell ts}^\omega$	场景 ω 下超出正常销量上限的产量	斤
Π_ω	场景 ω 下 2024–2030 年总名义利润	元
β	CVaR 置信水平, 本文取 $\beta = 0.90$	–
η	CVaR 线性化中的下尾阈值变量	元
ξ_ω	场景 ω 相对阈值 η 的下尾短缺量	元

5 数据预处理与基准数据构造

5.1 数据来源与字段统一

附件 1 给出地块与作物基础信息, 其中包括 54 个地块的名称、地类和面积, 以及 41 种作物的编号、名称和类别; 全部地块总面积为 1213 亩。附件 2 包含两类信息: 一类是 2023 年的实际种植记录, 另一类是不同作物在相应地类和季次下的亩产、种植成本与销售价格区间。后续模型需要同时使用地块属性、历史种植状态和经济参数, 因此首先按照统一的地块名称、作物编号、地类和季次字段对数据进行整理。

销售价格在附件中以区间给出。按照前述模型假设, 本文取区间中点作为 2023 年的代表性价格, 即

$$p_{j\ell s} = \frac{p_{j\ell s}^{\min} + p_{j\ell s}^{\max}}{2}. \quad (1)$$

该处理仅用于构造基准价格, 不将区间中点解释为实际成交均价。

5.2 种植记录清洗与参数连接

2023 年种植表中部分地块名称采用合并单元格表示, 读取后出现 33 条地块名为空的续行。由于这些空值只表示“沿用上一行地块”, 本文仅对地块名称列向下填充, 不对作物、种植面积和季次等字段进行填充, 避免将上一条种植记录误复制到下一行。

清洗后得到 87 条 2023 年种植记录。参数表共有 107 条“作物–地类–季次”记录, 两表以作物编号、地类和季次作为联合键进行连接。对于智慧大棚第一季的 6 条种植记录, 根据附件说明, 在参数查找时采用普通大棚第一季对应参数。连接完成后逐条检查参数映射结果, 未出现未匹配记录或一条种植记录对应多条参数记录的情况。

仅检查单条种植面积是否超过地块面积仍不足以保证容量一致性, 因为同一地块同一季次可能同时种植多种作物。因此, 本文进一步按“地块–季次”汇总种植面积, 并与地块容量比较。核验结果表明, 所有聚合后的种植面积均未超过对应地块容量。

5.3 2023 年基准量构造

记 2023 年种植记录集合为 $\mathcal{R}$ 。对任一记录 $r \in \mathcal{R}$ ，设其种植面积为 a_r ，对应作物、地类和季次分别为 $j(r)$ 、 $\ell(r)$ 和 $s(r)$ ，则理论产量和理论种植成本分别定义为

$$Q_r = a_r y_{j(r)\ell(r)s(r)}, \quad (2)$$

$$C_r = a_r c_{j(r)\ell(r)s(r)}. \quad (3)$$

由此可重构 2023 年的理论生产规模。全部 87 条记录汇总后，理论总产量为 1,260,505 斤，理论种植成本为 807,519 元。

附件没有直接给出 2023 年各作物的预期销量。为建立问题一及后续随机规划中的正常销售上限，本文将同一作物、同一季次的 2023 年理论产量汇总为销量代理：

$$D_{js} := \sum_{r \in \mathcal{R}: j(r)=j, s(r)=s} Q_r. \quad (4)$$

最终得到 47 个“作物-季次”基准销量记录。这里采用“作物-季次”而不是“作物总量”作为销量粒度，是因为题目的超产销售规则按作物及种植季次进行核算；同时，不再按地类拆分需求上限，可以避免同一作物的市场需求在不同地类间被重复计算。地类差异仍保留在亩产、成本和价格参数中，并在后续产销分配中继续体现。

需要强调的是，式 (4) 给出的是建模所需的销量代理，而非观测到的真实市场销量。库存、损耗及实际市场容量均可能造成理论产量与真实销量之间的偏差，因此后文将对超产处置规则单独进行敏感性分析。

5.4 数据一致性核验

为保证 Q1-Q3 使用同一套基准数据，本文在模型求解前完成结构与数值核验，主要结果见表 2。

表 2: 基准数据整理与核验结果

核验项目	结果
地块数量	54
地块总面积	1213 亩
作物数量	41
2023 年种植记录	87 条
参数表记录	107 条
合并单元格产生的地块名空续行	33 条
智慧大棚第一季参数映射记录	6 条
参数未匹配记录	0 条
一对多参数匹配记录	0 条
地块-季次容量超限	0 处
作物-季次销量基准	47 条
2023 年理论总产量	1,260,505 斤
2023 年理论种植成本	807,519 元

上述核验完成后，形成统一的地块表、作物表、2023 年种植事实表及“作物-地类-季次”参

数表。后续 Q1 直接使用该基准数据；Q2 和 Q3 只对题目允许变化的销量、亩产、成本和价格等参数生成情景，不再改变基础数据的连接关系和农业可行域。

5.5 土地结构特征

不同地类的面积分布及作物资格域如图 2 所示。梯田和平旱地面积分别为 619 亩和 365 亩，占全部土地的主要部分；普通大棚和智慧大棚合计仅 12 亩。面积占比与种植自由度并不等价：水浇地需要在单季水稻和两季蔬菜之间选择制度，大棚则具有明确的季次资格。后续模型因此保留逐地块决策与适种域约束，而不将 1213 亩土地合并为一个统一土地池。

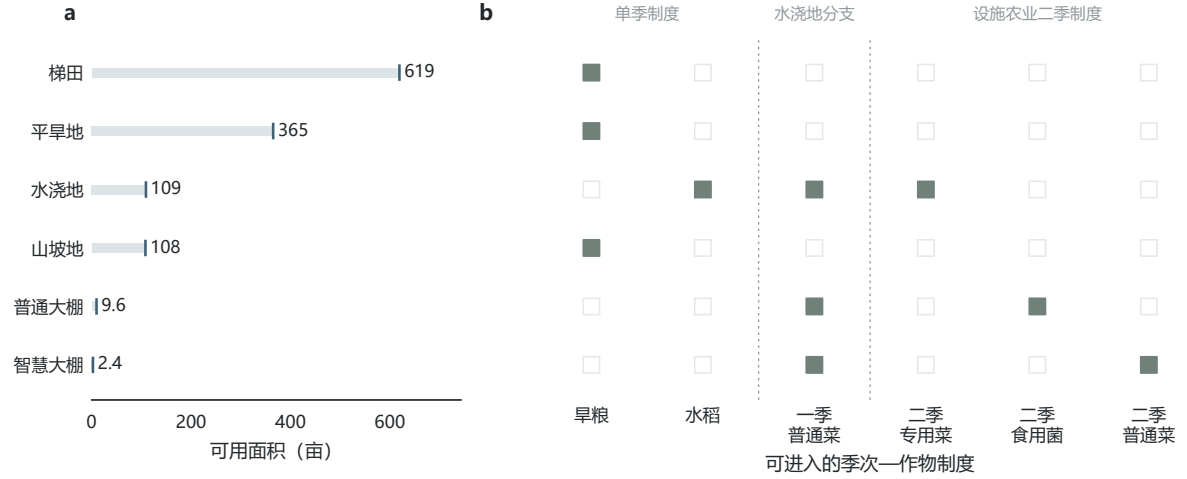


图 2: 土地面积与“地类—季次—作物组”资格矩阵。左图的细条给出各地类可用面积，右图的实心方格表示该组合属于代码定义的适种域。

6 问题一：确定性种植规划

6.1 建模思路

问题一要求在 2024–2030 年间给出逐地块、逐作物、逐季次的种植面积，并分别考虑“超产滞销”和“超产按当年价格的 50% 销售”两种规则。两种情形的农业条件完全相同，差别仅在超出正常销量上限后的收入。因此，本文先建立一套统一的农业可行域，再通过改变超产部分的价格系数得到两个可直接比较的确定性方案。

从决策过程看，种植面积决定产量，产量再经过正常市场和超产市场两条销售渠道形成收入。地类适种性、水浇地年度制度、相邻种植机会和豆类轮作共同限制种植面积。由于面积变量为连续变量，而种植启用状态和水浇地年度制度需要 0–1 变量表示，同时产销关系可写成线性形式，故问题一构成混合整数线性规划（MILP）。

6.2 产量、销量与利润函数

对作物 j 、地类 ℓ 、年份 t 和季次 s ，定义该组在所有对应地块上的总产量为

$$P_{j\ell ts} = y_{j\ell s} \sum_{\substack{i: \ell_i = \ell \\ (i,j,t,s) \in \mathcal{F}}} x_{ijts}. \quad (5)$$

其中, y_{jls} 为基准亩产, x_{ijts} 为种植面积。问题一中各经营参数保持在 2023 年基准水平, 因此 P_{jllts} 由种植决策唯一确定。

同一作物、同一季次的正常市场销量上限在不同地类之间共享。令 q_{jllts} 表示其中按正常价格销售的数量, 则有

$$0 \leq q_{jllts} \leq P_{jllts}, \quad (6)$$

$$\sum_{\ell} q_{jllts} \leq D_{js}. \quad (7)$$

式(6)保证正常销售量不能超过实际产量, 式(7)保证不同地类不能重复占用同一销量上限。由于销售价格均为非负数, 利润最大化会自动将正常销售量推至允许的最大范围, 因此无需显式写出 $\min\{P, D\}$ 这样的非线性表达。

超出正常市场容量的产量记为 e_{jllts} , 由产量守恒得到

$$e_{jllts} = P_{jllts} - q_{jllts}, \quad e_{jllts} \geq 0. \quad (8)$$

令 δ 表示超产部分相对于正常售价的折扣系数, 则问题一的两种规则可统一写为

$$\delta = \begin{cases} 0, & \text{超产部分滞销,} \\ 0.5, & \text{超产部分按正常售价的 50\% 销售.} \end{cases} \quad (9)$$

于是 2024–2030 年的总名义种植利润为

$$\max \Pi^{(\delta)} = \sum_{t \in \mathcal{T}} \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{J}} \sum_{\ell} \left[p_{jls} q_{jllts} + \delta p_{jls} e_{jllts} - c_{jls} \sum_{\substack{i: \ell_i = \ell \\ (i, j, t, s) \in \mathcal{F}}} x_{ijts} \right]. \quad (10)$$

该目标只计入题目提供的销售收入和种植成本, 因此本文将其称为“名义种植利润”, 不将其解释为包含运输、仓储、税费或折现后的会计净利润。

6.3 农业可行域

6.3.1 适种性、容量与启用关系

首先根据附件给出的地类–作物–季次适种关系构造可行集合 $\mathcal{F}$, 仅对 $(i, j, t, s) \in \mathcal{F}$ 建立种植变量。这样可以从变量域层面排除不允许的种植组合, 而不是在模型中大量加入“面积等于 0”的冗余约束。

对任一地块 i 、年份 t 和季次 s , 种植面积之和不得超过地块面积:

$$\sum_{j: (i, j, t, s) \in \mathcal{F}} x_{ijts} \leq A_i. \quad (11)$$

题目没有要求每块土地每季必须满种, 因此式(11)采用不等式, 允许在经济上不合算或受轮作限制时留出部分面积。

为描述某一作物是否在对应种植机会中出现, 引入 0–1 变量 z_{ijts} , 并建立面积与启用状态

之间的联结：

$$0 \leq x_{ijts} \leq A_i z_{ijts}, \quad z_{ijts} \in \{0, 1\}. \quad (12)$$

这里不设置 $x_{ijts} \geq \varepsilon z_{ijts}$ 形式的最小面积约束，因为题面只提出“单块种植面积不宜过小”，并未给出可核验的数值阈值。若人为指定 ε ，会使最终方案受主观参数影响。

6.3.2 水浇地年度种植制度

水浇地每年只能在“单季水稻”和“两季蔬菜”两种制度中选择一种，不能在同一地块同一年将部分面积安排水稻、剩余面积再安排两季蔬菜。令 $\mathcal{I}_W$ 为水浇地集合， $r_{it} = 1$ 表示选择单季水稻制度， $r_{it} = 0$ 表示选择两季蔬菜制度。分别记 $\mathcal{J}_R$ 、 $\mathcal{J}_{V1}$ 、 $\mathcal{J}_{V2}$ 为该地类下允许的水稻、第一季蔬菜和第二季蔬菜集合，则有

$$\sum_{j \in \mathcal{J}_R} x_{ijt,R} \leq A_i r_{it}, \quad (13)$$

$$\sum_{j \in \mathcal{J}_{V1}} x_{ijt,1} \leq A_i (1 - r_{it}), \quad (14)$$

$$\sum_{j \in \mathcal{J}_{V2}} x_{ijt,2} \leq A_i (1 - r_{it}), \quad (15)$$

$$r_{it} \in \{0, 1\}. \quad (16)$$

其中季次下标 $R, 1, 2$ 分别表示水浇地数据表中的单季水稻机会、第一季蔬菜机会和第二季蔬菜机会。上述约束控制的是整块土地的年度制度；制度内部仍允许多种兼容作物共享地块面积。

6.3.3 相邻种植机会与重茬约束

连续重茬的关键是“相邻物理种植机会”，而不只是年份是否相邻。对于一年一熟的露地，相邻机会通常位于相邻年份；对于一年两季的棚室或两季蔬菜制度，还需要连接同年第一季与第二季，以及上一年第二季与下一年第一季。

对地块 i ，按真实时间顺序建立物理种植机会集合 $\mathcal{H}_i$ 和相邻边集 $\mathcal{E}_i$ 。若机会 h 对应年份 $t_i(h)$ 和季次 $s_i(h)$ ，则对任意 $(h, h') \in \mathcal{E}_i$ 以及在两端均可种植的作物 j ，施加

$$z_{ij t_i(h) s_i(h)} + z_{ij t_i(h') s_i(h')} \leq 1. \quad (17)$$

式(17)直接禁止同一作物在相邻物理机会连续出现，从而在题目未给出具体重茬减产比例的情况下，以保守方式落实“避免连续重茬”的要求。

规划期不能从 2024 年孤立开始。若作物 j 出现在地块 i 的 2023 年最后一次实际种植机会中，则将该历史状态与 2024 年的第一次兼容机会相连，并禁止该作物立即重复种植。这样可以避免只检查 2024–2030 内部轮作而遗漏规划期边界。

6.3.4 滚动三年豆类约束

记 $\mathcal{J}_B$ 为豆类作物集合， B_i^{2023} 为地块 i 在 2023 年的实际豆类种植面积。题目要求每块土地在任意连续三年内至少安排一次豆类，但没有给出“种植一次”对应的最小面积。本文采用三年累计豆类面积不低于地块面积的线性近似。

首个三年窗口需要保留 2023 年的历史信息：

$$B_i^{2023} + \sum_{t=2024}^{2025} \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{J}_B} x_{ijts} \geq A_i. \quad (18)$$

对其余完整窗口 $u = 2024, \dots, 2028$ ，有

$$\sum_{t=u}^{u+2} \sum_{s \in \mathcal{S}} \sum_{j \in \mathcal{J}_B} x_{ijts} \geq A_i. \quad (19)$$

该近似可以保证三年内投入相当于一轮全地块面积的豆类种植量，同时避免人为引入“最小豆类面积”阈值。其局限在于只控制累计面积，不描述豆类在地块内部的空间位置，因此后文将其作为模型近似而非真实农田空间过程解释。

6.4 求解方法与结果

将式(10)分别取 $\delta = 0$ 和 $\delta = 0.5$ ，在完全相同的农业可行域上求解两次 MILP。两次求解均采用 0.5% 的 MIP 相对间隙作为停止容差。求解器均返回满足全部农业约束的可行方案及相应有效界，因此本文报告“停止容差内的可行方案”，不将非零间隙结果表述为严格的零间隙全局最优。

两种超产规则下的主要结果见表 3。

表 3: 问题一两种超产规则下的求解结果

方案	七年利润/万元	七年收入/万元	种植成本/万元	超产量/万斤	MIP 相对间隙
超产滞销	4104.45	4666.02	561.57	16.11	0.4141%
超产半价销售	6390.94	7052.60	661.66	949.19	0.03563%

为观察七年排程而不把数千个地块单元直接堆入正文，图 3按作物汇总半价方案的年度种植面积。小麦、玉米等主粮承担稳定的面积骨架，部分蔬菜和豆类则随轮作窗口发生年度调整。该图只做汇总展示；地块级完整计划仍保留在支撑材料中。

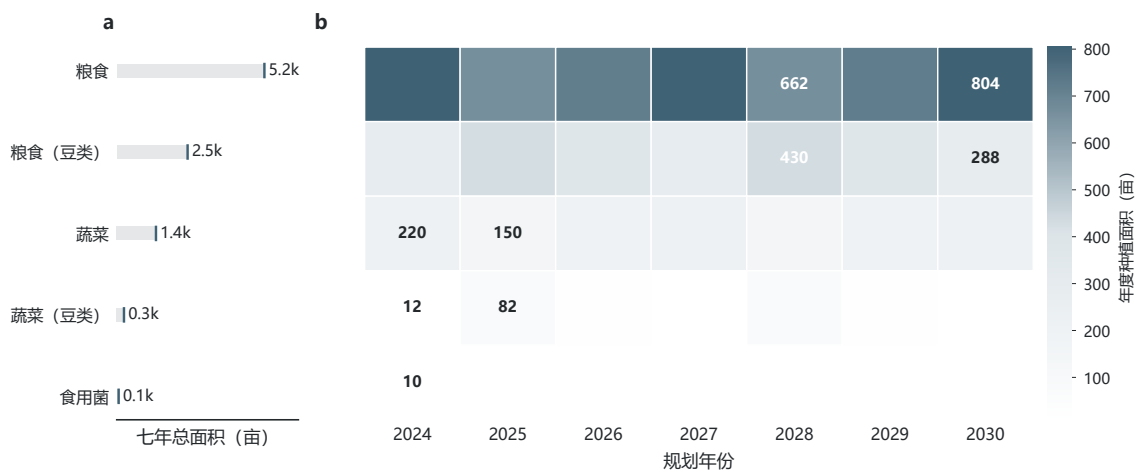


图 3: Q1 半价方案的年度-作物组种植面积。数值由全部地块、季次和作物按五类作物组汇总；色深表示年度种植面积，每行仅标注七年峰值与谷值。

图 4进一步拆开正常销量收入、超产折价收入与种植成本。半价方案的利润增长并非来自成本下降，而主要来自模型承认了规模很大的第二市场收入；这正是后续敏感性分析必须单独改变超产规则的原因。

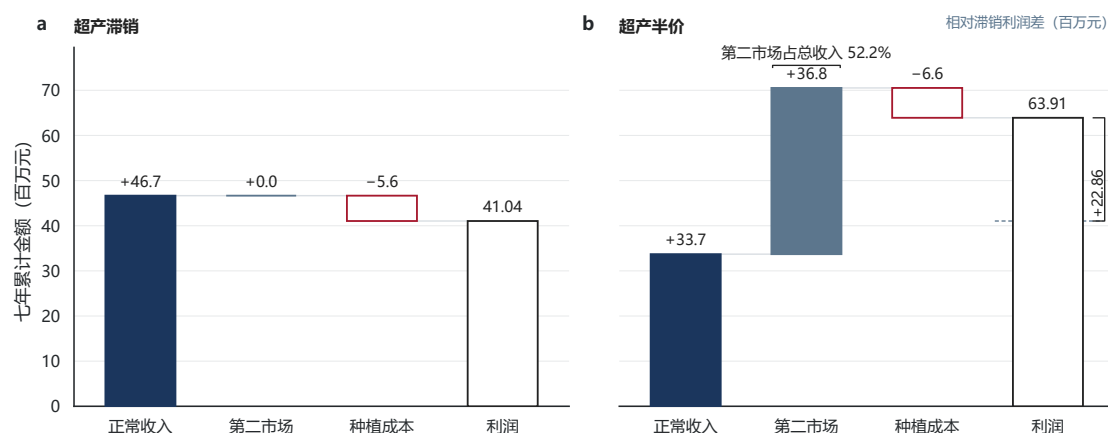


图 4: Q1 两种超产规则的收入、成本与利润分解。深蓝和板岩灰柱分别表示正常收入与第二市场收入，砖红边框空心柱表示由总收入向利润扣减的种植成本，黑色边框空心柱为最终利润；面板 (b) 的上方括号与右侧参考括号分别给出第二市场收入占比和相对滞销情形的利润差。

当超产部分完全滞销时，超过销量代理 D_{js} 的产量不再带来收入，模型因而倾向于使生产规模与正常市场容量相匹配。由于地类适种性、轮作和豆类约束具有离散性，超产无法被完全消除，但七年累计超产量仅为 16.11 万斤。

当超产部分仍可按正常售价的 50% 销售时，超出正常市场容量的产品仍可能具有正边际收益。模型因此更愿意扩大部分高产、高价值作物的种植面积，使七年名义利润由 4104.45 万元提高到 6390.94 万元，但累计超产量也由 16.11 万斤增加到 949.19 万斤。换言之，利润增加 2286.49 万元的同时，超产暴露增加 933.08 万斤。

这一差异不能简单理解为“半价销售方案一定更优”。第二种规则实际上假定存在一个容量未受限制、能够持续吸收全部超产产品的第二市场，而题目并未提供加工、储运和外销渠道的容量数据。因此，问题一的两个方案更适合作为两种市场边界：一端完全不承认超产价值，另一端假定全部超产均可按半价变现。真实经营环境位于两者之间时，超产产品的实际消化能力将直接影响最优种植结构和利润水平。

由此，问题一得到的不只是两组种植方案，还揭示了后续随机规划必须保留的一条关键经济机制：**正常销量上限决定第一市场容量，超产处置规则决定第二市场的边际价值**。问题二将在保持上述农业可行域不变的基础上，将销量、亩产、成本和价格由固定参数扩展为随机情景，并进一步评价固定种植方案在不利年份中的收益风险。

7 问题二：随机规划与下尾风险

问题一假定未来参数固定，因此同一套种植方案只对应一个利润值。问题二中，销量、亩产和部分价格随年份变化，同一套面积安排会对应一组不同的未来利润。此时真正需要解决的问题不再是“哪个方案在一组代表性参数下利润最高”，而是“哪个方案在多数可能情形下仍能保持较好的收益，并且在不利年份中不会出现过大的利润下滑”。

因此，本文保留问题一的全部农业可行域，只把未来经营参数替换为随机情景，并将期望利

润与利润分布的下尾风险同时纳入评价。为避免模型利用事后信息，种植面积、启用状态和水浇地年度种植制度都必须在情景实现之前确定；情景只影响产量、销量、价格和最终利润。这种“先决策、后观察随机结果”的非预见性结构与两阶段随机规划的基本思想一致 [2]。

7.1 随机参数与情景生成

题目给出的是未来参数的变化范围或变化趋势，而不是概率分布。本文据此先保留题面规定的变化规则，再在缺少进一步概率资料时，将有界随机扰动设为相应区间上的均匀分布。均匀分布只表示区间内部不额外偏向某一取值，并不意味着这些变量在现实中已经被证明服从均匀分布。

具体地，小麦和玉米的预期销量按年递增。记其年份 t 的增长率为 g_{jt}^ω ，则

$$D_{jst}^\omega = D_{js,t-1}^\omega (1 + g_{jt}^\omega), \quad g_{jt}^\omega \sim U(0.05, 0.10), \quad (20)$$

其中 $D_{js,2023} = D_{js}$ 。这种写法使增长率每年重新变化，但销量水平具有递推关系，符合“逐年增长”的题意。

对于其他作物，题目只给出相对 2023 年基准销量的年度波动范围，因此不把波动继续累乘成随机游走，而写为

$$D_{jst}^\omega = D_{js} (1 + u_{jt}^\omega), \quad u_{jt}^\omega \sim U(-0.05, 0.05), \quad (21)$$

其中上式中的随机扰动统一记为 u_{jt}^ω 。这样可以避免连续七年复合波动将题目给出的年度变化幅度放大成未经支持的长期趋势。

亩产同样围绕 2023 年参数波动。对作物 j 、地类 ℓ 和季次 s ，设

$$y_{j\ell ts}^\omega = y_{j\ell s} (1 + v_{jt}^\omega), \quad v_{jt}^\omega \sim U(-0.10, 0.10). \quad (22)$$

同一作物在同一年份的亩产扰动共享到其不同地类和已有季次基值上，从而既保留参数表中的地类、季次差异，又避免把同一种作物在同一年人为拆成互不相关的多个产量冲击。

成本和价格按照题目给出的趋势递推。种植成本记为

$$c_{j\ell ts} = c_{j\ell s} (1.05)^{t-2023}. \quad (23)$$

粮食类销售价格保持基准水平；蔬菜类销售价格每年按 5% 增长；普通食用菌价格的年度降幅在 1%–5% 之间随机变化；羊肚菌价格每年下降 5%。对 2023 年基准销量为 0 的“作物–季次”组合，本文不额外补造正需求，其正常市场容量仍取 0。

在上述边际规则下，问题二假设不同作物、年份和参数族的随机创新相互独立。该假设用于建立问题二的基准随机环境；作物之间的共同冲击与替代互补关系将在问题三中单独引入。

7.2 两阶段信息结构与情景利润

随机规划中最重要的约束之一是信息时序。2024–2030 年的种植计划需要在未来销量、亩产和价格尚未实现时作出，因此一阶段决策

$$x_{ijts}, \quad z_{ijts}, \quad r_{it}$$

均不带情景下标 ω 。如果给面积变量增加情景下标，就等于允许决策者提前知道每条未来路径后再决定种什么，这会把不可实现的完全信息收益混入模型。

在情景 ω 下，给定已经确定的种植面积后，理论产量为

$$P_{j\ell ts}^\omega = y_{j\ell ts}^\omega \sum_{i:\ell_i=\ell} x_{ijts}. \quad (24)$$

正常销量仍同时受产量和该情景下的市场容量限制：

$$0 \leq q_{j\ell ts}^\omega \leq P_{j\ell ts}^\omega, \quad \sum_{\ell} q_{j\ell ts}^\omega \leq D_{jst}^\omega. \quad (25)$$

超出正常市场容量的产量进入半价销售通道：

$$e_{j\ell ts}^\omega = P_{j\ell ts}^\omega - q_{j\ell ts}^\omega. \quad (26)$$

因此，情景 ω 下 2024–2030 年的总名义利润为

$$\Pi_\omega = \sum_{j,\ell,t,s} p_{j\ell ts}^\omega (q_{j\ell ts}^\omega + 0.5e_{j\ell ts}^\omega) - \sum_{i,j,t,s} c_{j\ell ts} x_{ijts}. \quad (27)$$

与问题一相比，农业约束完全不变，变化只来自情景参数和由此产生的利润分布。

7.3 样本平均近似

连续随机空间无法直接放入整数规划求解器，因此采用样本平均近似（Sample Average Approximation, SAA），用有限个代表性情景近似真实期望 [2]。若优化情景数为 N ，则样本内期望利润写为

$$\hat{\mathbb{E}}[\Pi] = \frac{1}{N} \sum_{\omega=1}^N \Pi_\omega. \quad (28)$$

主优化使用 $N = 60$ 个拉丁超立方情景，随机种子固定为 20240818。拉丁超立方抽样通过分层覆盖各输入变量的边际区间，在有限样本下可提高情景空间的覆盖效率 [3]，适合本题这类整数规划计算成本较高的情形。

60 个情景参与方案选择，因此不能再把同一批情景当作最终泛化评价。方案确定后，本文使用不同随机种子 20240819 重新生成 3000 条普通蒙特卡洛路径，只评价已经固定的种植方案，不再进行重新优化。这样做不能消除概率分布假设本身可能存在的偏差，但可以检验方案是否过度依赖于用于优化的那 60 个情景。

图 5 给出了 Q2 从随机输入到利润分布的经济流。种植面积在情景实现前固定；需求、亩产、价格和成本只改变各情景中的销量分配与经济核算，不允许模型在看到情景后重新调地。

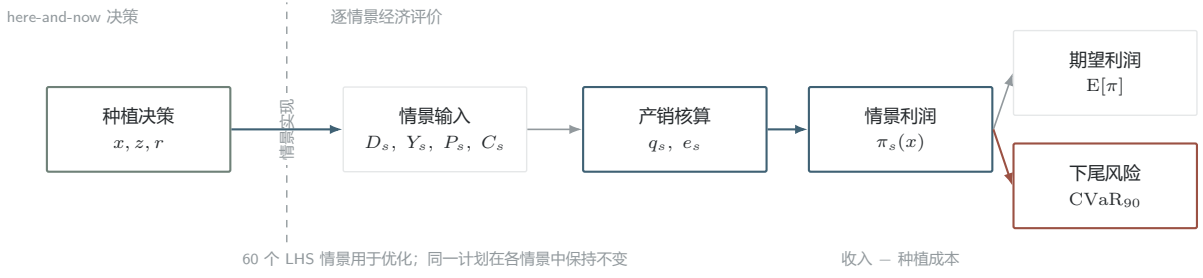


图 5: Q2 不确定参数、固定种植决策与利润风险指标之间的传递关系。

7.4 下尾 CVaR 风险度量

只最大化期望利润可能掩盖低概率但损失较大的年份，而只优化最坏情景又容易被单个极端样本支配。本文采用利润的 90% 下尾条件风险价值 (Conditional Value at Risk, CVaR)，刻画利润分布中最差 10% 情景的平均水平。CVaR 可通过引入辅助阈值与短缺变量转化为线性形式 [4]。

需要注意，这里度量的是“利润下尾”，因此符号方向与常见的“损失 CVaR”写法相反。取 $\beta = 0.90$ ，引入自由变量 η 和非负短缺变量 ξ_ω ：

$$\xi_\omega \geq \eta - \Pi_\omega, \quad \xi_\omega \geq 0, \quad (29)$$

则样本 CVaR 可写为

$$\widehat{\text{CVaR}}_\beta = \eta - \frac{1}{(1 - \beta)N} \sum_{\omega=1}^N \xi_\omega. \quad (30)$$

直观上， η 给出一个候选下尾阈值；只有当某个情景利润低于 η 时， ξ_ω 才记录短缺。最大化上述表达式，就会提高整段下尾利润，而不是只抬高平均值。对于 3000 条等权样本和 $\beta = 0.90$ ，外样本 CVaR 即为排序后最差 300 条利润的算术平均。

CVaR 只增加线性辅助变量与线性约束，因此可以直接嵌入问题一的 MILP 结构。本文把期望利润与 CVaR 分别作为“中心收益”和“下尾安全性”的指标，不把 CVaR 解释为“90% 概率保证达到的利润”。

7.5 双目标方案与近理想性检验

期望利润和 CVaR 具有相同量纲，但管理含义不同。若任意指定一个加权系数把二者直接相加，最终方案可能由人为权重主导。本文因此不依赖单个加权目标来声称得到完整 Pareto 前沿，而是对已经求得并通过农业可行性与经济复算的 Q2 候选方案，分别计算其到“最大期望利润”和“最大 CVaR”两个单目标上界的距离。

对期望利润，风险中性 MILP 的求解器有效界给出单目标上界；对 CVaR，保持 60 个情景、全部农业约束、经济守恒关系和 CVaR 定义不变，仅将 7490 个 0-1 变量放松为相同上下界内的连续变量。由于整数可行域包含在其连续松弛可行域中，连续松弛的最优 CVaR 必然是整数最大 CVaR 的上界。

若候选值为 V ，对应有效上界为 U ，定义相对差距

$$\text{Gap} = \frac{U - V}{|U|} \times 100\%. \quad (31)$$

Q2 候选在两个方向上的结果如表 4所示。

表 4: Q2 方案的样本内指标与单目标有效上界

指标	候选方案值/万元	有效上界/万元	相对差距
样本内期望利润	6964.06	6964.65	0.0084%
样本内 CVaR ₉₀	6831.29	6836.37	0.0744%

两个差距均低于 0.5%，说明该候选同时非常接近两个单目标理想值。因此本文将其称为“0.5% 近理想方案”。这一表述只说明方案同时接近两个有效上界，并不意味着完整 Pareto 前沿已经求出，也不意味着两个上界能够被同一个整数方案同时达到。

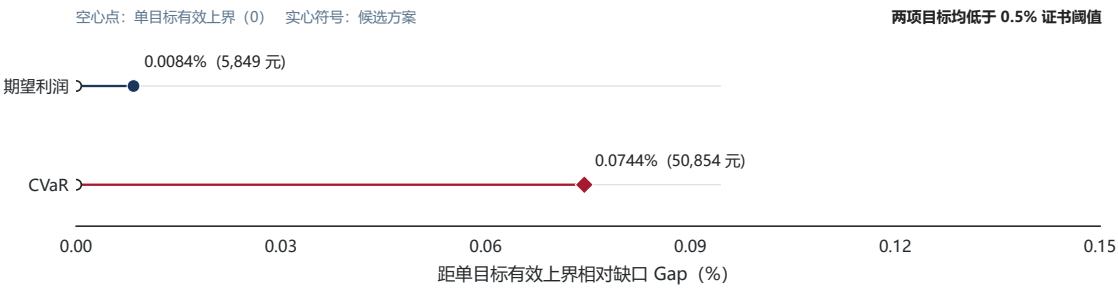


图 6: Q2 候选到期望利润和 CVaR 单目标有效上界的相对距离。横轴放大到 0–0.15% 以区分两个小缺口；两项缺口均低于 0.5% 证书阈值。

7.6 独立样本外评价

为了检验 Q2 方案是否只是在 60 个优化情景上表现较好，本文将 Q2 方案和问题一的两套固定方案放到同一组 3000 条独立路径上重新计算利润。三套方案使用完全相同的随机路径，因此方案之间的差异主要来自种植结构，而不是两批随机样本恰好一好一坏。

结果如表 5所示。

表 5: 3000 条独立样本外路径上的方案比较

方案	平均利润/万元	标准差/万元	5% 分位数/万元	10% 分位数/万元	CVaR ₉₀ /万元	最低利润/万元
Q2 近理想方案	6963.37	81.85	6828.52	6856.07	6820.53	6699.68
Q1 案例 2: 超产半价	6934.10	80.87	6801.20	6827.99	6793.00	6673.79
Q1 案例 1: 超产滞销	4216.58	20.31	4182.47	4190.12	4180.53	4146.52

Q2 相对 Q1 案例 2 的平均利润提高约 29.27 万元，CVaR₉₀ 提高约 27.53 万元，而两者标准差仅相差约 0.98 万元。也就是说，Q2 的改善并不是通过明显扩大总体波动来换取更高均值，而是在保持相近波动尺度的同时，使整个利润分布略向右移动，并同步抬高了最差 10% 路径的平均利润。

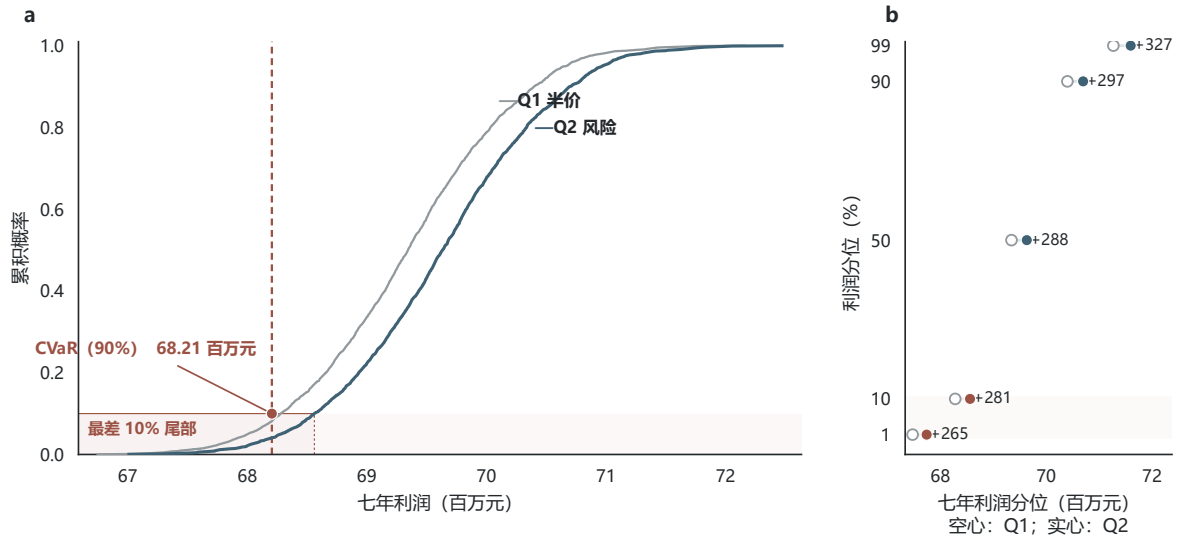


图 7: Q2 与 Q1 案例 2 的样本外利润分布及分位提升。左图使用同一组 3000 条路径绘制真实经验分布, 阴影为 Q2 最差 10% 区域; 右图给出 Q2 相对 Q1 案例 2 在不同利润分位上的提升。

从经验分布函数可以看到, 两套高利润方案的分布存在较大重合, 但 Q2 曲线整体略向右。右图进一步表明, 这种右移并不只出现在中位数附近, 也延伸到最差 1% 和 10% 的低利润路径。左图标出的尾部均值才是本文使用的下尾 CVaR。这与表 5 中的结论一致, 也说明 Q2 的改善主要体现在利润分布的位置和下尾水平, 而不是通过放大总体波动获得更高均值。

Q1 案例 1 在本节评价中利润明显较低, 并不能理解为其算法或农业排程质量较差。该方案原本是在“超产无收入”的规则下优化, 而这里为了公平比较固定种植结构, 统一按问题二的半价超产环境重新评价。它更适合作为后续超产市场敏感性分析中的保守基准。

综上, 在本文给定的独立随机环境下, Q2 方案同时在样本内接近期望利润与 CVaR 两个单目标上界, 并在独立样本外路径上略优于 Q1 半价方案。下一步仍需回答一个更重要的问题: 如果天气、市场和成本冲击会同时作用于多种作物, 这一结论是否仍然稳定。该问题将在问题三中进一步检验。

8 问题三：相关不确定性与作物间关系

问题二把不同作物、年份和参数族的随机创新视为相互独立。这一设定便于先看清各类边际波动的影响, 但它会忽略一种更现实的情况: 同一年份中, 多种作物往往会受到相同天气、市场景气或投入成本冲击。例如, 异常天气可能同时影响多种作物的亩产, 某一类农产品的市场景气可能同时影响组内需求与价格, 化肥和能源价格上涨也可能共同推高多种作物的种植成本。已有农业研究表明, 温度与水分等气象条件的耦合会共同影响作物产量, 而能源和化肥价格变化能够通过农业投入成本传导到食品生产与价格 [8, 9]。

因此, 问题三不改变问题二已经确定的农业可行域和“先决策、后实现”的信息结构, 但随机环境并非只改相关系数: 除共同因子和需求端价格反馈外, 代码还把 Q2 中固定的粮食、蔬菜价格增长率和成本增长率改为题面区间内的随机变量。Q2 与 Q3 仍可在同一 Q3 路径上公平比较种植方案, 但观察到的差异应归属于整个 Q3 相关经营环境, 不能单独识别为“相关性效应”。

8.1 为什么采用低维共同因子

附件只提供一个基准年份，无法利用历史时间序列直接估计 41 种作物之间的经验协方差矩阵。若人为给出一个 41×41 的稠密相关矩阵，不仅需要大量缺乏数据支撑的参数，还容易得到难以解释甚至不满足半正定要求的相关结构。

本文因此采用低维共同因子表示“同源波动”。设 F_t^W 表示年份 t 的天气共同因子， $F_{g,t}^M$ 表示作物组 g 的市场共同因子， F_t^C 表示投入成本共同因子，均取标准正态潜变量。对任一随机参数族 X ，使用统一形式

$$Z_X = \sum_k \lambda_{Xk} F_k + \sqrt{1 - \sum_k \lambda_{Xk}^2} \varepsilon_X, \quad (32)$$

其中 $\varepsilon_X \sim N(0, 1)$ 为对应的独立扰动， λ_{Xk} 为共同因子载荷。由于

$$\text{Var}(Z_X) = 1,$$

不同随机量通过共享因子获得相关性，同时每个潜变量仍保持标准正态边际。

随后通过标准正态分布函数 $\Phi(\cdot)$ 得到均匀分位

$$U_X = \Phi(Z_X), \quad (33)$$

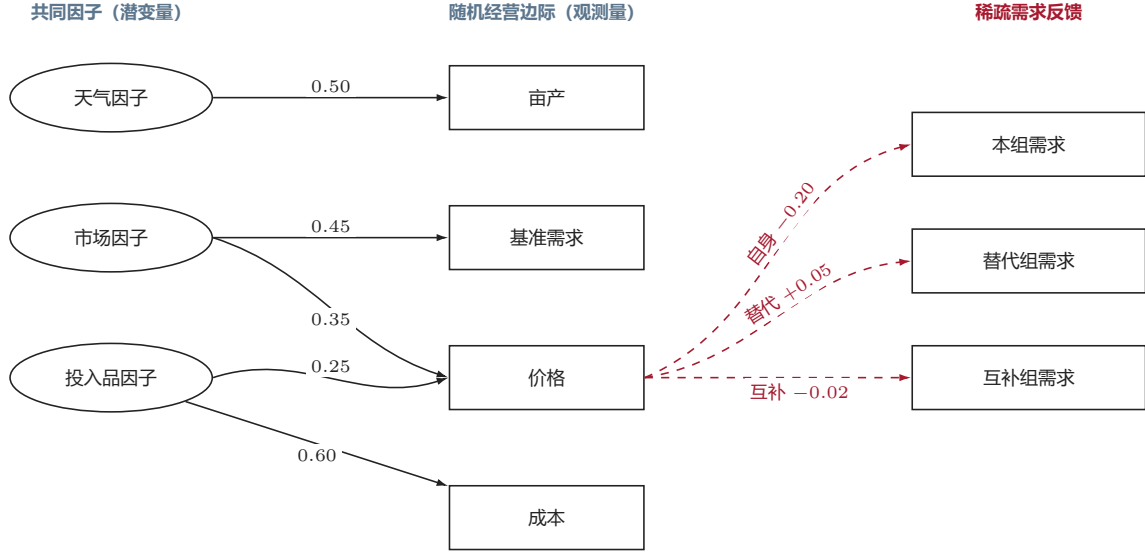
再把 U_X 映射到 Q3 配置中声明的有界边际分布。亩产和需求的问题区间与 Q2 保持一致；价格和成本则使用 Q3 代码中明确的问题区间，其中粮食价格年增幅为 $[-1\%, 1\%]$ 、蔬菜为 $[3\%, 7\%]$ 、成本为 $[3\%, 7\%]$ ，羊肚菌价格降幅为 $[3\%, 7\%]$ 。这些变量不超出设定区间，但其边际随机性较 Q2 的固定增长率更丰富。

共同因子只用于描述相关结构，不把它解释为天气、市场或投入成本对利润的因果效应。基准载荷如表 6 所示。由于缺少本地多年数据，这些载荷是机制导向的建模设定，而不是由该村历史样本估计得到的统计参数。

表 6: 问题三的共同因子载荷设定

共同作用关系	方向	基准载荷
天气因子 $\rightarrow$ 亩产	同向	0.50
组市场因子 $\rightarrow$ 需求	同向	0.45
组市场因子 $\rightarrow$ 价格	同向	0.35
投入因子 $\rightarrow$ 成本	同向	0.60
投入因子 $\rightarrow$ 价格	同向	0.25

这里的结构比直接指定大量两两相关系数更容易解释：同一作物组的需求与价格因为共享市场因子而共同波动，多种作物的亩产因为共享天气因子而产生同向变化，多种作物的成本则通过投入因子共同上升或下降。外部研究只用于支持影响方向和机制，不能替代本地参数估计，因此后文还需要对载荷强度进行敏感性分析。



三个共同因子替代 41×41 稠密相关矩阵；乘子截断 [0.90, 1.10]

载荷与弹性均为情景参数，不作本地因果估计

图 8: Q3 共同因子、随机经营边际与稀疏弹性之间的依赖结构。实线表示共同因子传递，虚线表示 E 类弹性反馈；后者用于构造相关经营情景，不作本地因果估计解释。

图 8 把问题三相对于问题二新增的结构集中在两处：共同因子使多个经营变量同向变化，稀疏弹性再把价格变化反馈到需求。两类关系最终都只改变情景利润的生成方式，种植面积仍须在情景实现前固定，因此 Q2 与 Q3 的差别可以继续放到同一组路径上比较。

8.2 稀疏的需求-价格交互

问题三还要求考虑不同作物之间可能存在的替代或互补关系。若对 41 种作物的所有两两组合都设置交叉价格弹性，需要估计超过千个关系系数，而附件没有提供相应需求数据。为避免把“考虑相关性”变成大规模参数猜测，本文只保留具有明确经济解释的稀疏关系。

41 种作物按经济和食用属性划分为干豆、谷物主粮、鲜食豆、薯类、果菜/瓜类、叶菜/十字花科、根菜和食用菌八组。需求交互遵循三个原则：第一，自身价格上涨通常抑制本作物需求；第二，替代关系只在同组作物之间出现；第三，仅在主粮与非淀粉蔬菜之间设置较弱的组级互补关系。食品需求研究普遍支持自身价格弹性为负，同时表明不同品类的交叉价格弹性可因替代或互补关系表现为正或负，且不同研究中的估计差异较大 [5, 6, 7]。因此，没有预先声明的作物对，其交叉弹性取 0。

记 p_{jt}^ω 为作物 j 在情景 ω 下的相关价格路径， p_{jt}^{ref} 为问题二对应的参考价格路径；记 $I_{g,-j,t}^\omega$ 为作物 j 所在组中其他作物的相对价格指数， $I_{h,t}^\omega$ 为与其存在组级互补关系的价格指数。需求交互因子写为

$$M_{jt}^\omega = \exp \left[\varepsilon_{\text{own}} \ln \frac{p_{jt}^\omega}{p_{jt}^{\text{ref}}} + \varepsilon_{\text{sub}} \ln I_{g,-j,t}^\omega + \varepsilon_{\text{comp}} \ln I_{h,t}^\omega \right]. \quad (34)$$

其中指数项使弹性可以直接解释为局部百分比响应，同时保证需求乘数为正。基准弹性取值见表 7。

表 7: 问题三的稀疏价格弹性设定

关系	方向	基准弹性
自身价格弹性	负	-0.20
组内替代弹性	正	+0.05
主粮-非淀粉蔬菜互补弹性	负	-0.02

这些点值只用于构造一个保守的局部反馈环境，不解释为本村经验估计值。为防止缺少本地校准时价格交互被无界放大，本文将需求乘数截断在

$$\widetilde{M}_{jt}^{\omega} = \min \{1.10, \max (0.90, M_{jt}^{\omega})\}, \quad (35)$$

最终需求上限写为

$$D_{jst}^{\omega, Q3} = D_{jst}^{\omega, \text{factor}} \widetilde{M}_{jt}^{\omega}. \quad (36)$$

也就是说，价格反馈最多在共同因子生成的基础需求附近再调整 $\pm 10\%$ 。这种处理牺牲了一部分外推能力，但可以避免在没有本地弹性估计的情况下让少数人为参数主导整个种植方案。

8.3 相关情景下的 Q3 候选方案

Q3 继续采用样本平均近似和 90% 下尾 CVaR，农业约束、一阶段决策变量和产销守恒关系均与 Q2 一致；情景生成器则同时实现共同因子、稀疏弹性和上述价格/成本区间。主优化使用 60 个相关拉丁超立方情景，随机种子固定为 20240820。

与 Q2 相同，本文分别比较候选方案与期望利润单目标有效上界、CVaR 单目标有效上界的距离。结果见表 8。

表 8: Q3 方案的样本内指标与单目标有效上界

指标	Q3 候选/万元	有效上界/万元	相对差距
样本内期望利润	6957.05	6964.86	0.1124%
样本内 CVaR ₉₀	6729.36	6742.49	0.1951%

两个差距仍低于 0.5%，说明 Q3 候选在当前相关情景样本中同时接近两个单目标上界。不过，这一结果只说明候选方案具有较好的近理想性，不能解释为已经求出了完整 Pareto 前沿或精确双目标全局最优解。

8.4 同一相关环境下比较 Q2 与 Q3

判断 Q3 是否值得替代 Q2，不能把“Q2 在独立环境下的表现”与“Q3 在相关环境下的表现”直接相比，因为那样同时改变了方案和评价分布。正确的比较方式是：固定 Q2 与 Q3 两套种植计划，并把它放在同一组相关路径上逐条评价。

本文使用随机种子 20240821 生成 3000 条相关蒙特卡洛路径，两套方案共享完全相同的路径。结果见表 9。

表 9: 3000 条相关路径上的 Q2 与 Q3 比较

方案	平均利润/万元	标准差/万元	5% 分位数/万元	CVaR ₉₀ /万元	最低利润/万元
Q2 方案	6965.11	118.80	6768.71	6755.36	6562.20
Q3 候选	6957.81	119.07	6760.14	6747.65	6550.86

两套利润分布非常接近，但 Q2 的平均利润和 CVaR 均略高；逐路径比较中，超过 99% 的相关路径上 Q2 利润高于 Q3 候选。因此，在当前共同因子与弹性设定下，重新按 Q3 优化并没有获得可验证的样本外收益优势。

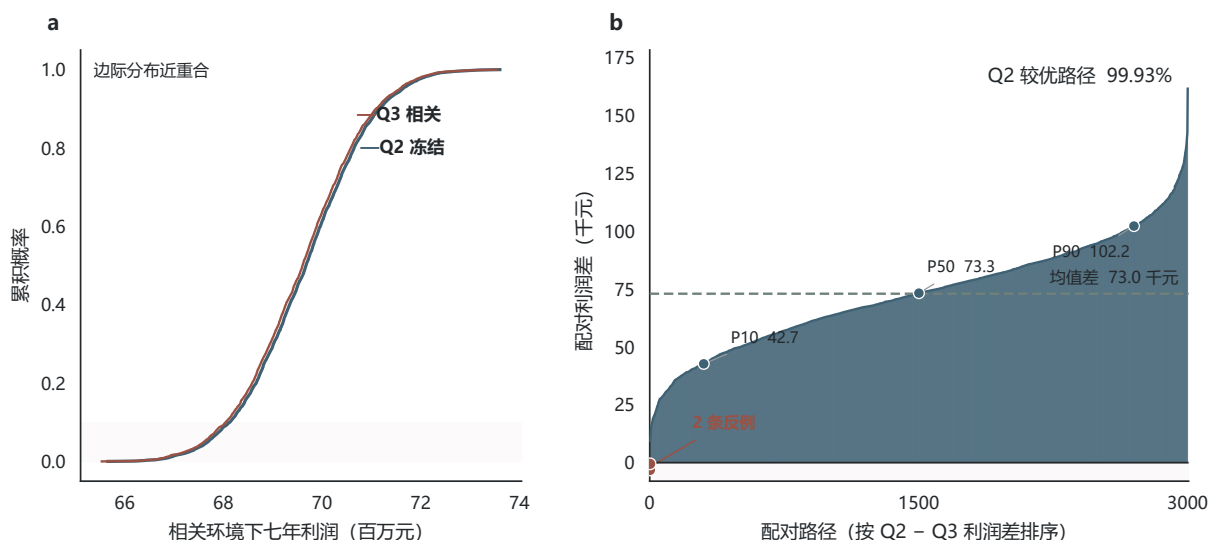


图 9: 同一 Q3 相关路径下 Q2 与 Q3 的利润分布及逐路径利润差

但 Q3 仍揭示了一个重要风险事实。Q2 方案在独立环境下的样本外 CVaR 为 6820.53 万元，而同一 Q2 方案放入 Q3 环境后降为 6755.36 万元；平均利润则基本保持在 6960 万元附近。由于两个环境同时存在依赖结构和部分边际设定差异，这个跨环境数值本身不能把降幅全部归因于共同冲击。后文在 Q3 边际规则不变时单独改变共同因子强度，仍观察到依赖增强持续压低 CVaR，才为“共同冲击主要加重下尾风险”提供了可分离的敏感性证据。

8.5 种植结构是否发生实质变化

为了区分“利润分布变了”和“种植方案本身变了”，本文进一步比较 Q2 与 Q3 的种植面积结构。按作物汇总 2024–2030 年的累计种植面积后，两方案的相关系数为

$$\rho_{\text{area}} = 0.999608, \quad (37)$$

归一化作物总面积 L_1 差约为

$$\Delta_{L_1}^{\text{norm}} \approx 2.209\%. \quad (38)$$

从作物总量层面看，两套方案几乎保持相同的主要作物组合。与此同时，在地块–作物–年份–季次的细粒度层面，共有 583 个单元发生变化。这说明 Q3 的调整主要体现在“在哪里种、哪一年种、哪个季次种”，而不是把“种什么”整体推翻。

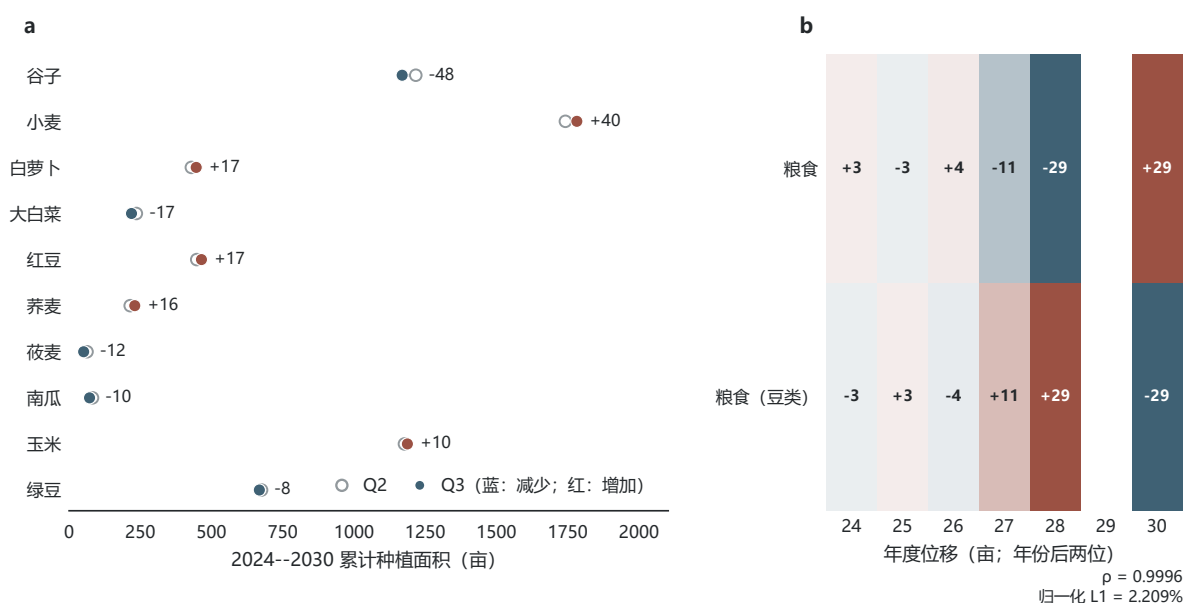


图 10: Q2 与 Q3 的种植结构差异。左图列出七年累计面积位移绝对值最大的 10 种作物, 空心点为 Q2、实心点为 Q3; 右图给出作物组-年份面积变化。

因此, 问题三得到的结论不是“相关模型越复杂, 方案就一定越好”。在当前 Q3 参数范围内, 相关经营环境加重了下尾风险, 但没有提供调整主要作物组合的充分依据; Q3 候选本身也没有在统一 Q3 路径上超过 Q2。基于现有证据, 更合理的做法是继续把 Q2 作为种植计划的实施基准, 同时利用 Q3 评估共同冲击下的风险边界, 并通过后续敏感性分析判断这一结论是否依赖于载荷和弹性参数的具体取值。

9 模型检验与敏感性分析

前述结果建立在若干数据代理和随机设定之上, 因此仅报告最优目标值不足以说明方案可靠。本节从两个层面检验结论: 首先检查最终种植计划是否同时满足农业约束和经济核算关系; 随后改变最可能影响决策的关键设定, 观察方案排序和风险指标是否发生实质变化。敏感性分析均采用**固定方案重评**: 改变外部环境后重新计算既有方案的利润, 而不重新优化种植面积。这样测得的是原方案对假设变化的暴露程度, 避免把“环境变化”和“重新选择方案”混在一起。

9.1 农业可行性与经济核算一致性

对 Q1-Q3 的最终面积计划, 分别从农业约束和经济核算两条路径进行复查。农业层逐项检查地块适种性、地块容量、水浇地年度制度、相邻物理种植机会、2023 年边界状态以及滚动三年豆类约束; 随机规划还额外检查所有情景是否共享同一组一阶段种植决策。上述检查的目的不是再次求解模型, 而是确认输出的面积表本身确实落在前文定义的农业可行域中。

经济层则从种植面积重新计算产量、正常销量、超产量、销售收入、种植成本和七年利润, 并据场景利润重新计算期望利润与经验 CVaR。这样可以避免仅依赖求解器内部目标值: 若面积计划、产销守恒或风险统计中的任一环节存在口径不一致, 最终复算结果就会暴露差异。

对于整数规划求解结果, 本文同时报告可行方案值和求解器有效界。前者说明当前方案可以实现的目标水平, 后者用于限定未知最优值可能达到的上限; 二者角色不同, 因此不把“达到停止容差”表述为“证明得到精确全局最优”。Q1 两种情形的相对间隙分别为 0.4141% 和 0.03563%,

Q2 候选到期望利润和 CVaR 两个单目标有效上界的差距分别为 0.0084% 和 0.0744%，Q3 对应差距为 0.1124% 和 0.1951%。这些数值说明当前候选已足以支撑后续方案比较，但仍保留整数优化本身的数值边界。

9.2 超产处理规则的敏感性

问题一已经表明，超产能否进入折价市场会显著改变生产激励。问题二和问题三虽然采用“超产按 50% 价格销售”作为主情形，但现实中的第二市场容量、折价程度和销售损耗均未由题目给出，因此该设定需要单独检验。

我们把超产折价系数由 0.5 改为 0，只重新评价已经得到的 Q1 与 Q2 固定方案。在问题二的独立随机环境中，方案排序发生反转：半价销售时 Q2 的样本外平均利润和 CVaR 分别为 6963.37 万元和 6820.53 万元；当超产全部滞销时，Q1 案例 1 反而成为平均利润和 CVaR 最高的方案，对应数值分别为 4096.90 万元和 4064.39 万元。由此可见，Q2 相对 Q1 案例 1 的优势并不是对超产市场假设不敏感，而是建立在第二市场仍具有正价值的条件下。

在问题三的相关随机环境中也得到相同方向的结果：超产半价时，固定 Q2 方案的平均利润和 CVaR 分别为 6965.11 万元和 6755.36 万元；改为全部滞销后，两项指标降至 2718.34 万元和 2670.69 万元。虽然在同一相关环境、同一规则下，固定 Q2 仍略优于 Q3 候选，但绝对利润水平大幅下降。这说明当前决策中最值得进一步核实的并不是更复杂的求解算法，而是第二市场究竟能够吸收多少超产、实际折价曲线如何以及是否存在储运和合同约束。

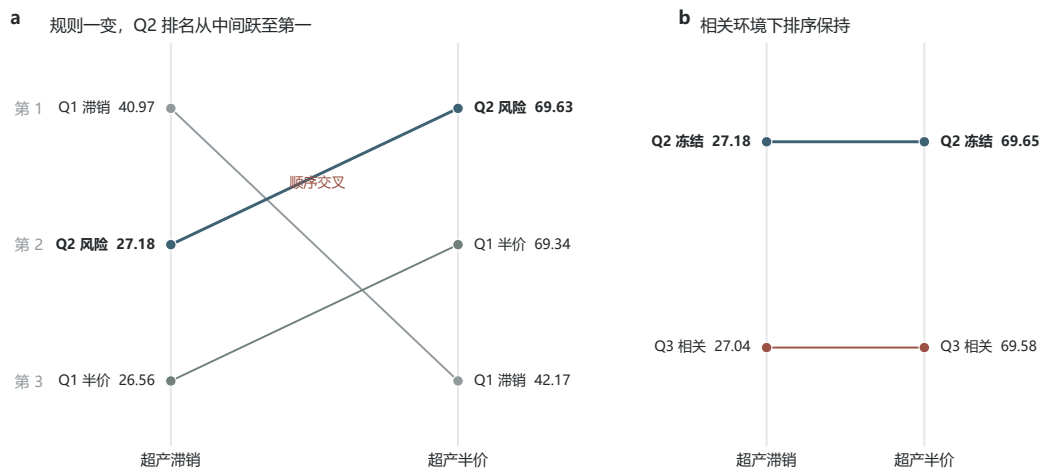


图 11: 不同超产处理规则下固定方案的平均利润与列内排名。端点数字为同一评价环境下的七年平均利润（百万元），连线交叉表示离散规则切换后推荐顺序发生改变。

图 11 的作用是比较同一批固定方案在不同超产规则下的表现，而不是再次寻找各规则下的新最优解。若排序随折价规则改变，说明该经济假设足以影响方案推荐；这一结论比单纯比较某个模型的样本内目标值更重要。

9.3 共同冲击强度与价格弹性敏感性

Q3 中的共同因子载荷和价格弹性没有当地长时间序列用于直接估计，因此本文进一步改变两类参数，检验“共同冲击主要影响尾部、而不明显改变主要种植结构”这一结论是否只依赖某组点值。

具体地，将共同因子强度设为弱、基准、强三档，同时将价格弹性整体乘以 0、1 和 1.5 三个倍率，形成 3×3 共 9 组评价环境。这里仍使用固定 Q2 与 Q3 方案进行重评，不在每个参数组合下重新优化。这样能够直接观察同一方案在依赖程度和需求价格反馈改变时的风险暴露。

结果显示，共同冲击增强时，平均利润基本保持稳定，而 CVaR 持续下降。以 Q3 候选为例，在基准弹性下，其平均利润约保持在 6957.8 万元附近，而 CVaR 由弱依赖环境下约 6772.5 万元下降至强依赖环境下约 6720.5 万元。也就是说，相关性的主要作用不是显著改变利润中心，而是使不利情形更容易同时发生，从而加厚利润分布的下尾。

相比之下，将价格弹性从 0 倍提高到 1.5 倍所造成的变化较小。一方面，本文采用的自身、替代和互补弹性本身较为保守；另一方面，需求乘数被限制在 90%–110% 范围内。因此这里只能得到“在当前局部且受限的交互设定下，弹性影响相对较小”的结论，不能据此推断真实市场中的替代与互补关系普遍不重要。

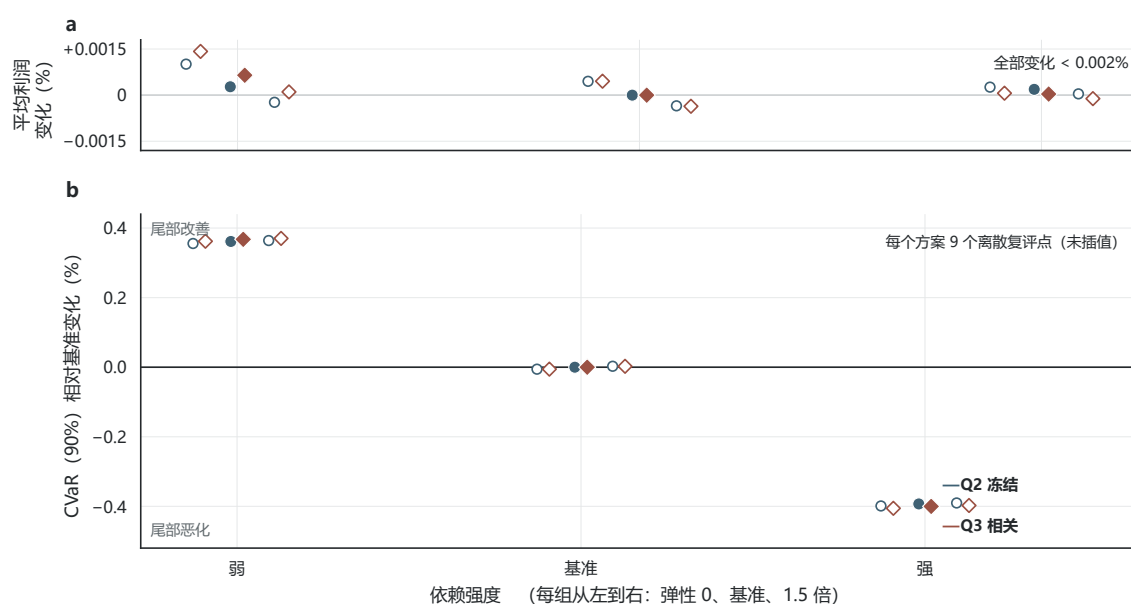


图 12: 固定 Q2、Q3 种植方案在共同冲击强度与价格弹性 3×3 组合下的离散复评。每个方案的九个点均为既定情景设定下的独立计算结果，未进行插值，也未在新参数下重新优化种植方案。

若图 12 中期望利润在不同依赖强度下变化较小，而 CVaR 随共同冲击增强明显下降，则与 Q3 样本外比较得到的机制一致：相关结构主要修正尾部风险认识，而不是自动生成更高收益的种植方案。

9.4 敏感性分析结论

综合上述检验可以把关键设定分成两类。第一类是会改变方案选择的经济条件，其中超产产品的市场吸收能力最为突出；当超产由半价销售改为完全滞销时，问题二环境中的方案排序直接反转。第二类是主要改变风险评价而较少改变种植结构的参数，共同冲击强度属于这一类：它对平均利润影响有限，却持续压低 CVaR。价格弹性在当前保守取值和需求截断下影响较小，但由于缺少当地需求数据，仍应视为需要后续校准的参数。

因此，本文后续的方案建议不把任何一个随机模型视为脱离假设的“绝对最优”。Q2 可以作为当前参数设定下的主要种植基准，而 Q3 更适合用来衡量相关冲击下的尾部风险；若现实中的第二市场容量或折价规则与主设定明显不同，则应首先重新评估方案排序。

10 模型评价与进一步讨论

前文从确定性规划、独立随机环境和相关随机环境三个层次逐步扩展模型，并通过样本外评价和敏感性分析检验方案表现。本节不再增加新的优化结构，而是结合现有结果说明模型的适用范围、主要局限以及进一步改进所需要的数据条件。

10.1 模型的主要特点

(1) **农业可行域在三个问题中保持一致。**本文没有因为引入随机性而改变地块适种性、水浇地种植制度、相邻种植机会和豆类轮作等农业规则。Q1、Q2 和 Q3 的差异主要来自经营参数是否随机以及随机变量之间是否存在共同冲击，因此不同模型的结果可以在同一农业约束基础上比较。这种处理避免了“模型越复杂，约束口径也随之改变”所带来的不可比性。

(2) **不确定性按照现实机制逐层加入。**Q1 先给出确定性基准，用于识别超产处理规则对利润和生产激励的影响；Q2 在此基础上引入销量、亩产、成本和价格的不确定性，并同时考察期望利润与下尾 CVaR；Q3 进一步增加共同因子和稀疏价格弹性，同时把部分价格、成本的固定年增长率改为题面区间内的随机变量。因此，Q2 与 Q3 的统一路径比较可以评价两套方案在同一 Q3 环境中的优劣，但不能把全部差异识别为纯粹的相关性效应；依赖强度的单独影响由固定 Q3 边际下的敏感性分析判断。

(3) **优化与评价相互分离。**Q2 使用 60 个拉丁超立方情景进行方案选择，方案确定后再使用新的 3000 条路径进行样本外评价；Q3 同样在独立生成的相关路径上比较固定 Q2 与 Q3 方案。因而，样本外结果反映的是既定种植计划在新情景中的表现，而不是重新利用评价样本调整方案。对 Q1–Q3 的利润和风险指标还进行了独立复算，并结合求解器有效界判断当前候选距离单目标理想水平的大小。

(4) **模型不仅比较收益，也识别结论成立的条件。**敏感性分析表明，第二市场的超产处理规则足以改变问题二环境中的方案排序；相比之下，共同冲击强度主要影响利润分布的下尾，而对平均利润和主要作物组合影响较小。因此，模型的作用不只是给出一张 2024–2030 年的面积表，也用于判断哪些外部条件真正值得在实施前优先核实。

10.2 模型的局限性

首先，附件没有提供 2023 年各作物的真实预期销量，本文以 2023 年种植记录和亩产参数计算得到的理论产量作为“作物–季次”销量代理。该代理能够保证数据口径闭合，但它描述的是基于生产端信息重构的市场容量，并不能替代真实的订单量、历史销量和渠道容量。因此，涉及正常市场上限和超产规模的结果都受到这一代理设定影响。

其次，销售单价使用附件价格区间的中点作为 2023 年代表性价格，利润中只计入销售收入和附件给出的种植成本，未加入题面没有提供的储运、人工组织、销售损耗等额外成本。尤其在半价销售情形下，模型相当于默认超产部分能够按给定折价持续销售，而敏感性分析已经表明，这一条件会明显影响方案排序。因此，当前利润应理解为在题面成本口径和超产规则下的名义种植利润，而不是对实际现金收益的完整预测。

再次，部分农业要求只能进行线性近似。题目没有给出“单块种植面积不宜过小”的明确阈值，因此主模型没有人为加入最小种植面积；豆类轮作则用三年累计豆类面积不低于地块面积进行近似。这些处理避免了凭空补造参数，但也意味着模型更擅长确定“种什么、种多少和何时种”，对田间作业便利性、单块内部空间分布等操作层细节刻画较弱。

最后，Q2 和 Q3 中的概率结构受数据条件限制。Q2 的均匀分布与独立性是缺少历史概率资料时的基准设定；Q3 中的共同因子载荷和价格弹性也是模型参数，而不是由该村多年数据估计得到的统计量。因此，Q3 能够回答“如果存在这类共同冲击，方案和尾部风险会怎样变化”，但不能把当前参数下的相关结构解释为对当地真实因果关系的识别。

10.3 进一步改进方向

现有结果已经表明，并非所有数据缺口都同等重要。若要把模型用于实际滚动决策，首先应补充作物-季次层面的真实销量、订单量、超产去向及折价成交数据。这些信息可以直接校准正常市场容量和第二市场收益，也是当前最可能改变方案排序的一组参数。

其次，应持续记录逐年亩产、天气、销售价格和投入成本。当积累多个年份后，可以用实际样本重新估计波动范围和变量间相关性，替代目前的均匀分布和预设共同因子载荷；若能够进一步获得作物价格与销量的联合历史数据，再对自身、替代和互补弹性进行估计，Q3 中的需求交互关系才可以从情景假设升级为数据校准模型。

第三，如果实际管理部门能够给出最小可操作种植面积、单个地块允许的作物数量、轮作执行成本和设施转换成本，这些信息可以直接加入混合整数规划，使数学上的可行方案进一步接近田间作业方案。此时模型的重点不必改变，只需在现有农业可行域上补充更细的管理约束。

在数据能够逐年更新的情况下，还可以把当前一次性制定 2024–2030 年计划的方式改为滚动更新：每年根据最新销量、价格、亩产和成本重新估计随机参数，在保留长期轮作要求的同时修正后续年份的种植安排。这样既保留长期规划的约束结构，又能减少七年参数全部由 2023 年基准外推所带来的不确定性。

总体而言，本文模型的主要价值在于把农业可行性、市场容量和收益风险放在同一框架下比较，并通过样本外评价和敏感性分析区分“稳定的结构性结论”和“依赖关键假设的方案排序”。其适用边界也较明确：当前结果可用于题目给定数据条件下的方案比较和风险分析，但若直接用于实际经营决策，应优先补充市场销售与多年经营数据，再据此校准随机参数和实施约束。

11 结论与建议

本文围绕 2024–2030 年山区乡村农作物种植规划问题，在统一农业可行域上依次建立确定性混合整数规划、考虑下尾风险的随机规划以及含共同冲击和稀疏价格弹性的相关风险模型。三层模型的差别主要在于对未来不确定性的描述逐步加深，因此各阶段结果可以在同一农业约束体系下直接比较。

11.1 主要结论

- (1) 超产处理方式会明显改变利润水平和生产激励。在问题一的确定性环境下，超产滞销和超产按 50% 价格销售两种规则对应的七年累计利润分别为 4104.45 万元和 6390.94 万元，超产量分别为 16.11 万斤和 949.19 万斤。后者利润更高，但同时明显依赖折价市场吸收大量超产产品，因此不能只根据名义利润判断方案优劣。
- (2) 在题目给定的独立不确定环境和半价超产规则下，Q2 方案具有更好的收益–风险综合表现。在 60 个拉丁超立方情景上，Q2 方案的样本内期望利润和 $CVaR_{90}$ 分别为 6964.06 万元和 6831.29 万元，距相应单目标有效上界仅 0.0084% 和 0.0744%。在另外生成的

3000 条独立样本外路径上，其平均利润和 $CVaR_{90}$ 分别为 6963.37 万元和 6820.53 万元，均高于问题一的半价销售方案。因此，在当前概率设定与销售规则下，Q2 更适合作为 2024-2030 年的基准种植方案。

- (3) **Q3 环境没有提供调整主要作物组合的证据；共同冲击主要加重尾部风险。**问题三加入天气、作物组市场和投入成本共同因子，用稀疏价格弹性描述部分作物间的替代与互补关系，并把部分价格、成本增长率改为区间随机变量。在同一组 3000 条 Q3 路径上，固定 Q2 方案的平均利润和 $CVaR_{90}$ 分别为 6965.11 万元和 6755.36 万元，Q3 候选分别为 6957.81 万元和 6747.65 万元；两方案各作物七年累计种植面积的相关系数达到 0.999608。由此可见，当前 Q3 参数下重新优化未带来更好的样本外表现，主要作物组合也基本稳定；在 Q3 边际规则不变的敏感性比较中，增强共同因子强度会持续压低 $CVaR$ ，说明依赖主要影响利润下尾。
- (4) **第二市场条件是当前结论中最需要优先核实的外部假设。**敏感性分析表明，当超产产品由半价销售改为全部滞销后，问题二环境中的方案排序发生反转，Q1 案例 1 在平均利润和 $CVaR$ 上均优于 Q2。相比之下，提高共同冲击强度主要压低 $CVaR$ ，而对平均利润影响较小；价格弹性在当前保守取值和需求截断范围内的影响也相对有限。因此，超产产品能否稳定进入第二市场，比继续增加模型复杂度更可能直接改变最终种植决策。

11.2 实施建议

基于上述结果，本文提出以下实施建议。

- (1) 在超产产品能够以接近主设定的折价水平持续销售时，可采用 Q2 方案作为 2024-2030 年的基准种植安排，并按照附件要求逐地块落实地类适种性、水浇地制度、重茬限制和豆类轮作要求。
- (2) 在实际执行前，应优先调查各类作物的正常市场容量、超产收购渠道、可实现折价、储运损耗和订单约束。若第二市场容量有限，不宜直接套用“全部超产均可半价销售”的情形，应根据实际可售数量重新评价现有方案。
- (3) 对天气异常、同类农产品需求同步下降以及投入品价格上涨等共同冲击，应保留 Q3 中的相关风险评价作为压力测试工具。其作用主要是判断既定方案在不利环境下可能承受的尾部损失，而不是在当前证据下替代 Q2 成为新的种植基准。
- (4) 七年规划不宜一次确定后长期不变。随着新的销量、成交价格、亩产、天气和投入成本数据逐年积累，可在保持长期轮作约束连续性的前提下滚动更新随机参数并重新求解后续年份，从而逐步减少当前销量代理、分布假设和弹性点值带来的不确定性。

综上，本文的核心结论并不是寻找一个脱离条件的“唯一最优方案”，而是在明确农业约束和市场假设的前提下，给出能够被复算、比较和压力检验的种植决策。当前设定下 Q2 的综合表现最好；真正可能改变这一推荐的关键因素，是超产产品的实际市场吸收能力及其折价方式。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于问题分解、代码辅助与调试、结果核验、图表生成和语言整理，详细使用情况见支撑材料。

参考文献

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2024 年全国大学生数学建模竞赛 C 题及附件 [Z]. 2024.
- [2] Shapiro A, Dentcheva D, Ruszczyński A. *Lectures on Stochastic Programming: Modeling and Theory*[M]. 2nd ed. Philadelphia: Society for Industrial and Applied Mathematics, 2014. DOI: 10.1137/1.9781611973433.
- [3] McKay M D, Beckman R J, Conover W J. A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code[J]. *Technometrics*, 1979, 21(2): 239–245. DOI: 10.1080/00401706.1979.10489755.
- [4] Rockafellar R T, Uryasev S. Optimization of Conditional Value-at-Risk[J]. *The Journal of Risk*, 2000, 2(3): 21–41. DOI: 10.21314/JOR.2000.038.
- [5] U.S. Department of Agriculture, Economic Research Service. Commodity and Food Elasticities: Documentation[EB/OL]. Updated 2025-01-08. <https://www.ers.usda.gov/data-products/commodity-and-food-elasticities/documentation>. Accessed 2026-08-18.
- [6] Han T, Wahl T I. China's Rural Household Demand for Fruit and Vegetables[J]. *Journal of Agricultural and Applied Economics*, 1998, 30(1): 141–150. DOI: 10.1017/S1074070800008129.
- [7] Andreyeva T, Long M W, Brownell K D. The Impact of Food Prices on Consumption: A Systematic Review of Research on the Price Elasticity of Demand for Food[J]. *American Journal of Public Health*, 2010, 100(2): 216–222. DOI: 10.2105/AJPH.2008.151415.
- [8] Lesk C, Coffel E, Winter J, Ray D, Zscheischler J, Seneviratne S I, Horton R. Stronger Temperature–Moisture Couplings Exacerbate the Impact of Climate Warming on Global Crop Yields[J]. *Nature Food*, 2021, 2(9): 683–691. DOI: 10.1038/s43016-021-00341-6.
- [9] Alexander P, Arneth A, Henry R, Maire J, Rabin S, Rounsevell M D A. High Energy and Fertilizer Prices Are More Damaging Than Food Export Curtailment from Ukraine and Russia for Food Prices, Health and the Environment[J]. *Nature Food*, 2023, 4(1): 84–95. DOI: 10.1038/s43016-022-00659-9.

A 支撑材料文件清单与复现方法

论文电子版与支撑材料为两个独立文件。下表列出匿名支撑包的实际目录：核心复现环境与开发期源文件分开存放，论文数字和图件均保留冻结依据。

表 10: 支撑材料文件列表

文件或目录	内容
model/	附件、2023 基线、建模决策、Q1–Q3 核心程序、冻结方案、依赖列表与统一复现入口。
development_source/	开发期运行入口、输出格式化和独立验证源文件；不参与论文排版。
results/	最终面积计划、求解指标、3000 条样本外路径和 36 组敏感性结果。
figures/、figure_data/ scripts/、 figures_tikz/、 figures_native/	F01–F12 的 PDF、SVG、PNG 及逐图派生数据。 统计图、TikZ 结构图与正文原生 PGFPlots 图的重建程序。
AI 工具使用详情.pdf	AI 工具用途、采纳情况、人工修改与核验记录。
supporting_material_ manifest.md	压缩包内实际文件名、字节数和 SHA-256。

解压支撑材料后，在其根目录执行：

```
python model/scripts/reproduce_core.py --paths 3000
```

程序按论文固定的随机种子重新生成样本外路径，检查 Q2、Q3 冻结方案的农业可行性，复算期望利润、CVaR 和配对差异；加入 `--solve-q1` 可重新求解问题一的两种情形。快速环境检查可把 `--paths` 改为 30。完整依赖版本见 `model/requirements.txt`。

B 核心建模程序选编

考虑到电子论文的可读性，本附录只全文列出统一复现入口、三个问题的核心优化模型以及 Q3 情景生成程序。这五个文件用于说明模型如何落到程序中，但不单独构成运行依赖闭包；其余数据读取、参数配置和独立验证模块均按原始目录保存在支撑材料的 `model/` 中。支撑包中的统一入口可直接运行，源码未截断、未使用省略号。

表 11: 纸面附录所列核心源码及 SHA-256

文件	SHA--256
source/scripts/reproduce_core.py	adda28fb878e3dcf536add4797478cb61937cd2f926ff943ff544b505369d542
source/src/q1_model.py	47665edbd4535d6cf272c01497cfedc7e7c9c385d2233aea8302909ff1758a
source/src/q2_model.py	f40269c003c40dd1b810b15a3857e76cabaa94c4bb57f67afa60eddf2a6e2c4
source/src/q3_scenarios.py	71f6e7a2e95dad1ec4e9e9aff9c4b363cec07b76aa30471771b850f859e88147
source/src/q3_model.py	d78af1b9fe5175094566e3e6f6fb43618e07d4de5bcc360848ee4fd66ae59de5

B.1 统一复现入口

Listing 1: `source/scripts/reproduce_core.py`

```
from __future__ import annotations

import argparse
import json
import math
```

```

import sys
from pathlib import Path

import numpy as np

MODEL_ROOT = Path(__file__).resolve().parents[1]
SRC = MODEL_ROOT / "src"
if str(SRC) not in sys.path:
    sys.path.insert(0, str(SRC))

from q1_data import load_q1_data # noqa: E402
from q1_model import build_q1_model, solve_q1_model # noqa: E402
from q2_config import DEFAULT_Q2_CONFIG # noqa: E402
from q2_uncertainty import generate_scenarios, lower_tail_cvar # noqa: E402
from q2_validate import evaluate_plan, load_plan_csv, validate_agricultural_plan # noqa: E402
from q3_config import BASELINE_Q3_CONFIG # noqa: E402
from q3_scenarios import generate_q3_scenarios # noqa: E402
from q3_validate import evaluate_q3_plan # noqa: E402

def summarize(values: np.ndarray, beta: float = 0.90) -> dict[str, float]:
    return {
        "mean_profit_yuan": float(np.mean(values)),
        "cvar_90_yuan": float(lower_tail_cvar(values, beta=beta)),
        "minimum_profit_yuan": float(np.min(values)),
        "maximum_profit_yuan": float(np.max(values)),
    }

def evaluate_frozen_plans(paths: int) -> dict[str, object]:
    data = load_q1_data(MODEL_ROOT)
    q2_plan = load_plan_csv(MODEL_ROOT / "results" / "q2_final_plan.csv")
    q3_plan = load_plan_csv(MODEL_ROOT / "results" / "q3_final_plan.csv")
    q2_feasibility = validate_agricultural_plan(data, q2_plan)
    q3_feasibility = validate_agricultural_plan(data, q3_plan)
    if not q2_feasibility.passed or not q3_feasibility.passed:
        raise RuntimeError("Frozen plan failed an agricultural feasibility check")

    q2_paths = generate_scenarios(
        data,
        count=paths,
        seed=DEFAULT_Q2_CONFIG.evaluation_seed,
        sampling_method=DEFAULT_Q2_CONFIG.evaluation_sampling,
        config=DEFAULT_Q2_CONFIG,
    )
    q2_eval = evaluate_plan(
        data,
        q2_plan,
        q2_paths,
        surplus_discount=DEFAULT_Q2_CONFIG.surplus_discount,
        config=DEFAULT_Q2_CONFIG,
    )

    q3_paths = generate_q3_scenarios(
        data,
        count=paths,
        seed=BASELINE_Q3_CONFIG.evaluation_seed,
        sampling_method="mc",
        config=BASELINE_Q3_CONFIG,
    )
    q2_in_q3 = evaluate_q3_plan(data, q2_plan, q3_paths, surplus_discount=0.5)
    q3_in_q3 = evaluate_q3_plan(data, q3_plan, q3_paths, surplus_discount=0.5)
    paired = q2_in_q3.profits_yuan - q3_in_q3.profits_yuan

    return {
        "paths": paths,
        "q2_environment": {
            "seed": DEFAULT_Q2_CONFIG.evaluation_seed,
            "sampling": DEFAULT_Q2_CONFIG.evaluation_sampling,
            "q2_plan": summarize(q2_eval.profits_yuan),
        },
        "q3_correlated_environment": {
            "seed": BASELINE_Q3_CONFIG.evaluation_seed,
            "sampling": "mc",
            "q2_plan": summarize(q2_in_q3.profits_yuan),
            "q3_plan": summarize(q3_in_q3.profits_yuan),
            "q2_better_path_count": int(np.sum(paired > 0)),
            "paired_mean_difference_yuan": float(np.mean(paired)),
        },
    }

def solve_q1() -> dict[str, object]:
    data = load_q1_data(MODEL_ROOT)
    output: dict[str, object] = {}

```

```

for name, discount in (("case1_waste", 0.0), ("case2_half_price", 0.5)):
    model = build_q1_model(data, case_name=name, excess_price_factor=discount)
    solution = solve_q1_model(model, time_limit_seconds=300.0, mip_relative_gap=0.005)
    if not bool(solution.result.success) or not math.isfinite(solution.total_profit_yuan):
        raise RuntimeError(f"Q1 solve failed: {name}")
    output[name] = {
        "profit_yuan": solution.total_profit_yuan,
        "mip_gap": float(getattr(solution.result, "mip_gap", math.nan)),
        "solver_status": int(solution.result.status),
    }
return output

def main() -> int:
    parser = argparse.ArgumentParser(description="Reproduce the paper's core model checks from the anonymous support package.")
    parser.add_argument("--paths", type=int, default=3000, help="Evaluation paths; use 30 for a quick smoke run.")
    parser.add_argument("--solve-q1", action="store_true", help="Also rerun the two Q1 MILPs.")
    parser.add_argument("--output", type=Path, default=MODEL_ROOT / "reproduction_summary.json")
    args = parser.parse_args()
    if args.paths <= 0:
        raise SystemExit("--paths must be positive")
    result: dict[str, object] = {"frozen_plan_evaluation": evaluate_frozen_plans(args.paths)}
    if args.solve_q1:
        result["q1_resolve"] = solve_q1()
    args.output.write_text(json.dumps(result, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(json.dumps({"status": "PASS", "output": str(args.output), "paths": args.paths}, ensure_ascii=False))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

B.2 问题一：确定性混合整数规划

Listing 2: source/src/q1_model.py

```

"""Sparse deterministic MILP for CUMCM 2024 Problem C, Question 1."""

from __future__ import annotations

import math
from collections import Counter, defaultdict
from dataclasses import dataclass
from typing import Hashable, Iterable, Mapping

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, OptimizeResult, milp
from scipy.sparse import coo_matrix

from q1_data import (
    IRRIGATED_LAND,
    SEASON_FIRST,
    SEASON_SECOND,
    SEASON_SINGLE,
    YEARS,
    Q1Data,
)

class Q1ModelError(RuntimeError):
    """Raised when the Q1 model cannot be built or solved to optimality."""

PlantKey = tuple[str, int, int, str]
RegimeKey = tuple[str, int]
SalesKey = tuple[int, str, int, str]
NUMERICAL_ZERO = 1e-8

def _clean_continuous(value: float) -> float:
    """Suppress solver-scale signed zeros without changing material decisions."""
    return 0.0 if abs(value) <= NUMERICAL_ZERO else float(value)

@dataclass(frozen=True)
class VariableBlock:
    x: dict[PlantKey, int]
    z: dict[PlantKey, int]
    regime: dict[RegimeKey, int]
    normal_sales: dict[SalesKey, int]
    excess_sales: dict[SalesKey, int]
    names: tuple[str, ...]
    lower_bounds: np.ndarray

```

```

upper_bounds: np.ndarray
integrality: np.ndarray
objective: np.ndarray

@dataclass(frozen=True)
class ConstraintSystem:
    linear_constraint: LinearConstraint
    names: tuple[str, ...]
    category_counts: dict[str, int]
    matrix_nonzeros: int

@dataclass(frozen=True)
class QIBuiltModel:
    data: QIData
    case_name: str
    excess_price_factor: float
    variables: VariableBlock
    constraints: ConstraintSystem

@dataclass(frozen=True)
class QISolution:
    model: QIBuiltModel
    result: OptimizeResult
    values: np.ndarray
    total_profit_yuan: float

    @property
    def x(self) -> dict[PlantKey, float]:
        return {
            key: _clean_continuous(self.values[index])
            for key, index in self.model.variables.x.items()
        }

    @property
    def z(self) -> dict[PlantKey, float]:
        return {key: float(self.values[index]) for key, index in self.model.variables.z.items()}

    @property
    def regimes(self) -> dict[RegimeKey, float]:
        return {
            key: float(self.values[index])
            for key, index in self.model.variables.regime.items()
        }

    @property
    def normal_sales(self) -> dict[SalesKey, float]:
        return {
            key: _clean_continuous(self.values[index])
            for key, index in self.model.variables.normal_sales.items()
        }

    @property
    def excess_sales(self) -> dict[SalesKey, float]:
        return {
            key: _clean_continuous(self.values[index])
            for key, index in self.model.variables.excess_sales.items()
        }

class VariableRegistry:
    def __init__(self) -> None:
        self.names: list[str] = []
        self.lower: list[float] = []
        self.upper: list[float] = []
        self.integrality: list[int] = []
        self.objective: list[float] = []

    def add(
        self,
        name: str,
        *,
        lower: float = 0.0,
        upper: float = math.inf,
        integer: bool = False,
        objective: float = 0.0,
    ) -> int:
        index = len(self.names)
        self.names.append(name)
        self.lower.append(float(lower))
        self.upper.append(float(upper))
        self.integrality.append(1 if integer else 0)
        self.objective.append(float(objective))
        return index

```

```

class SparseConstraintBuilder:
    def __init__(self, variable_count: int) -> None:
        self.variable_count = variable_count
        self.row_indices: list[int] = []
        self.column_indices: list[int] = []
        self.values: list[float] = []
        self.lower: list[float] = []
        self.upper: list[float] = []
        self.names: list[str] = []
        self.categories: Counter[str] = Counter()

    def add(
        self,
        coefficients: Mapping[int, float] | Iterable[tuple[int, float]],
        *,
        lower: float = -math.inf,
        upper: float = math.inf,
        category: str,
        name: str,
    ) -> None:
        row = len(self.lower)
        items = coefficients.items() if isinstance(coefficients, Mapping) else coefficients
        nonzero_count = 0
        for column, value in items:
            coefficient = float(value)
            if abs(coefficient) <= 0.0:
                continue
            self.row_indices.append(row)
            self.column_indices.append(int(column))
            self.values.append(coefficient)
            nonzero_count += 1
        if nonzero_count == 0:
            if lower <= 0 <= upper:
                return
            raise Q1ModelError(f"Infeasible empty constraint {name}: {lower} <= 0 <= {upper}")
        self.lower.append(float(lower))
        self.upper.append(float(upper))
        self.names.append(name)
        self.categories[category] += 1

    def build(self) -> ConstraintSystem:
        matrix = coo_matrix(
            (self.values, (self.row_indices, self.column_indices)),
            shape=(len(self.lower), self.variable_count),
            dtype=float,
        ).tocsr()
        return ConstraintSystem(
            linear_constraint=LinearConstraint(
                matrix,
                np.asarray(self.lower, dtype=float),
                np.asarray(self.upper, dtype=float),
            ),
            names=tuple(self.names),
            category_counts=dict(sorted(self.categories.items())),
            matrix_nonzeros=int(matrix.nnz),
        )

def sales_groups(data: Q1Data) -> tuple[SalesKey, ...]:
    groups = {
        (crop_id, data.plots[plot].land_type, year, season)
        for plot, crop_id, year, season in data.compatible_cells
    }
    return tuple(sorted(groups, key=lambda key: (key[2], key[3], key[0], key[1])))

def build_variables(data: Q1Data, excess_price_factor: float) -> VariableBlock:
    if excess_price_factor not in {0.0, 0.5}:
        raise Q1ModelError(f"Unsupported excess price factor: {excess_price_factor}")
    registry = VariableRegistry()
    x: dict[PlantKey, int] = {}
    z: dict[PlantKey, int] = {}
    regime: dict[RegimeKey, int] = {}
    normal_sales: dict[SalesKey, int] = {}
    excess_sales: dict[SalesKey, int] = {}

    for key in data.compatible_cells:
        plot, crop_id, year, season = key
        plot_data = data.plots[plot]
        parameter = data.parameters[(crop_id, plot_data.land_type, season)]
        x[key] = registry.add(
            f"x[{plot},{crop_id},{year},{season}]",
            lower=0.0,
            upper=plot_data.area_mu,

```

```

        objective=parameter.cost_yuan_per_mu,
    )
    z[key] = registry.add(
        f"z[{plot},{crop_id},{year},{season}]",
        lower=0.0,
        upper=1.0,
        integer=True,
    )

for plot in data.irrigated_plots:
    for year in YEARS:
        regime[(plot, year)] = registry.add(
            f"u_rice[{plot},{year}]", lower=0.0, upper=1.0, integer=True
        )

for key in sales_groups(data):
    crop_id, land_type, year, season = key
    parameter = data.parameters[(crop_id, land_type, season)]
    price = parameter.midpoint_price_yuan_per_jin
    normal_sales[key] = registry.add(
        f"normal[{crop_id},{land_type},{year},{season}]",
        lower=0.0,
        objective=-price,
    )
    excess_sales[key] = registry.add(
        f"excess[{crop_id},{land_type},{year},{season}]",
        lower=0.0,
        objective=-excess_price_factor * price,
    )

return VariableBlock(
    x=x,
    z=z,
    regime=regime,
    normal_sales=normal_sales,
    excess_sales=excess_sales,
    names=tuple(registry.names),
    lower_bounds=np.asarray(registry.lower, dtype=float),
    upper_bounds=np.asarray(registry.upper, dtype=float),
    integrality=np.asarray(registry.integrality, dtype=np.uint8),
    objective=np.asarray(registry.objective, dtype=float),
)

def add_capacity_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    cells_by_plot_season: dict[tuple[str, int, str], list[int]] = defaultdict(list)
    for (plot, _crop, year, season), index in variables.x.items():
        cells_by_plot_season[(plot, year, season)].append(index)
    for plot, year, season in data.available_plot_seasons:
        builder.add(
            ((index, 1.0) for index in cells_by_plot_season[(plot, year, season)]),
            upper=data.plots[plot].area_mu,
            category="capacity",
            name=f"capacity[{plot},{year},{season}]",
        )

def add_activation_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    for key, x_index in variables.x.items():
        plot = key[0]
        builder.add(
            {x_index: 1.0, variables.z[key]: -data.plots[plot].area_mu},
            upper=0.0,
            category="activation",
            name=f"activation[{plot},{key[1]},{key[2]},{key[3]}",
        )

def add_irrigated_regime_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    x_by_plot_year_season: dict[tuple[str, int, str], list[int]] = defaultdict(list)
    for (plot, _crop, year, season), index in variables.x.items():
        x_by_plot_year_season[(plot, year, season)].append(index)
    for plot in data.irrigated_plots:
        area = data.plots[plot].area_mu
        for year in YEARS:
            u = variables.regime[(plot, year)]
            builder.add(
                [(index, 1.0) for index in x_by_plot_year_season[(plot, year, SEASON_SINGLE)]]
                + [(u, -area)],
                upper=0.0,
            )

```

```

        category="irrigated_regime",
        name=f"rice_regime[{plot},{year}]",
    )
    for season in (SEASON_FIRST, SEASON_SECOND):
        builder.add(
            [(index, 1.0) for index in x_by_plot_year_season[(plot, year, season)]]
            + [(u, area)],
            upper=area,
            category="irrigated_regime",
            name=f"vegetable_regime[{plot},{year},{season}]",
        )

def add_irrigated_second_season_choice(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    for plot in data.irrigated_plots:
        for year in YEARS:
            indices = [
                variables.z[(plot, crop_id, year, SEASON_SECOND)]
                for crop_id in data.compatible_crops(plot, SEASON_SECOND)
            ]
            builder.add(
                ((index, 1.0) for index in indices),
                upper=1.0,
                category="irrigated_second_choice",
                name=f"irrigated_second_choice[{plot},{year}]",
            )

def add_rotation_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    for edge in data.adjacency_edges:
        first_crops = set(data.compatible_crops(edge.plot, edge.season_from))
        second_crops = set(data.compatible_crops(edge.plot, edge.season_to))
        for crop_id in sorted(first_crops.intersection(second_crops)):
            first_key = (edge.plot, crop_id, edge.year_from, edge.season_from)
            second_key = (edge.plot, crop_id, edge.year_to, edge.season_to)
            builder.add(
                {variables.z[first_key]: 1.0, variables.z[second_key]: 1.0},
                upper=1.0,
                category="rotation",
                name=(
                    f"rotation[{edge.plot},{crop_id},{edge.year_from},{edge.season_from},"
                    f"{edge.year_to},{edge.season_to}]"
                ),
            )

def add_initial_rotation_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    for plot, crop_ids in data.initial_last_crops.items():
        for season in data.initial_first_seasons[plot]:
            compatible = set(data.compatible_crops(plot, season))
            for crop_id in sorted(crop_ids.intersection(compatible)):
                key = (plot, crop_id, 2024, season)
                builder.add(
                    {variables.z[key]: 1.0},
                    upper=0.0,
                    category="initial_rotation",
                    name=f"initial_rotation[{plot},{crop_id},{season}]",
                )

def add_bean_window_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    bean_ids = data.bean_crop_ids
    for plot, plot_data in data.plots.items():
        first_indices = [
            index
            for (cell_plot, crop_id, year, _season), index in variables.x.items()
            if cell_plot == plot and crop_id in bean_ids and year in {2024, 2025}
        ]
        required_after_2023 = plot_data.area_mu - data.bean_area_2023[plot]
        builder.add(
            ((index, 1.0) for index in first_indices),
            lower=required_after_2023,
            category="bean_window",
            name=f"bean_window[{plot},2023-2025]",
        )
    for start in range(2024, 2029):
        indices = [
            index

```

```

        for (cell_plot, crop_id, year, _season), index in variables.x.items()
        if cell_plot == plot
        and crop_id in bean_ids
        and start <= year <= start + 2
    ]
    builder.add(
        ((index, 1.0) for index in indices),
        lower=plot_data.area_mu,
        category="bean_window",
        name=f"bean_window[{plot},{start}-{start + 2}]",
    )

def add_production_balance_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    x_by_group: dict[SalesKey, list[tuple[int, float]]] = defaultdict(list)
    for (plot, crop_id, year, season), index in variables.x.items():
        land_type = data.plots[plot].land_type
        parameter = data.parameters[(crop_id, land_type, season)]
        x_by_group[(crop_id, land_type, year, season)].append(
            (index, -parameter.yield_jin_per_mu)
        )
    for key, normal_index in variables.normal_sales.items():
        coefficients = x_by_group[key] + [
            (normal_index, 1.0),
            (variables.excess_sales[key], 1.0),
        ]
    builder.add(
        coefficients,
        lower=0.0,
        upper=0.0,
        category="production_balance",
        name=f"production_balance[{key[0]},{key[1]},{key[2]},{key[3]}]",
    )

def add_expected_sales_constraints(
    builder: SparseConstraintBuilder, data: Q1Data, variables: VariableBlock
) -> None:
    normal_by_crop_season_year: dict[tuple[int, int, str], list[int]] = defaultdict(list)
    for (crop_id, _land_type, year, season), index in variables.normal_sales.items():
        normal_by_crop_season_year[(crop_id, year, season)].append(index)
    for (crop_id, year, season), indices in sorted(normal_by_crop_season_year.items()):
        demand = data.expected_sales_proxy.get((crop_id, season), 0.0)
        builder.add(
            ((index, 1.0) for index in indices),
            upper=demand,
            category="expected_sales",
            name=f"expected_sales[{crop_id},{year},{season}]",
        )

def build_constraints(data: Q1Data, variables: VariableBlock) -> ConstraintSystem:
    builder = SparseConstraintBuilder(len(variables.names))
    add_capacity_constraints(builder, data, variables)
    add_activation_constraints(builder, data, variables)
    add_irrigated_regime_constraints(builder, data, variables)
    add_irrigated_second_season_choice(builder, data, variables)
    add_rotation_constraints(builder, data, variables)
    add_initial_rotation_constraints(builder, data, variables)
    add_bean_window_constraints(builder, data, variables)
    add_production_balance_constraints(builder, data, variables)
    add_expected_sales_constraints(builder, data, variables)
    return builder.build()

def build_q1_model(
    data: Q1Data, *, case_name: str, excess_price_factor: float
) -> Q1BuiltModel:
    variables = build_variables(data, excess_price_factor)
    constraints = build_constraints(data, variables)
    return Q1BuiltModel(
        data=data,
        case_name=case_name,
        excess_price_factor=excess_price_factor,
        variables=variables,
        constraints=constraints,
    )

def solve_q1_model(
    model: Q1BuiltModel,
    *,
    time_limit_seconds: float = 300.0,
    mip_relative_gap: float = 0.005,

```

```

        display_solver_log: bool = False,
    ) -> Q1Solution:
        result = milp(
            c=model.variables.objective,
            integrality=model.variables.integrality,
            bounds=Bounds(model.variables.lower_bounds, model.variables.upper_bounds),
            constraints=model.constraints.linear_constraint,
            options={
                "disp": display_solver_log,
                "presolve": True,
                "time_limit": float(time_limit_seconds),
                "mip_rel_gap": float(mip_relative_gap),
            },
        )
        if result.status != 0 or result.x is None or result.fun is None:
            raise Q1ModelError(
                f"{model.case_name} did not solve within the declared MIP gap: "
                f"status={result.status}, success={result.success}, message={result.message}"
            )
        values = np.asarray(result.x, dtype=float)
        if not np.all(np.isfinite(values)):
            raise Q1ModelError(f"{model.case_name} returned non-finite variable values")
        return Q1Solution(
            model=model,
            result=result,
            values=values,
            total_profit_yuan=-float(result.fun),
        )

```

B.3 问题二：独立场景与 CVaR 优化

Listing 3: source/src/q2_model.py

```

"""Sparse finite-scenario MILP for CUMCM 2024 Problem C, Question 2."""

from __future__ import annotations

import math
from dataclasses import dataclass, replace

import numpy as np
from scipy.optimize import Bounds, OptimizeResult, milp

from q1_data import Q1Data, YEARS
from q1_model import (
    NUMERICAL_ZERO,
    PlantKey,
    RegimeKey,
    SalesKey,
    SparseConstraintBuilder,
    VariableRegistry,
    add_activation_constraints,
    add_bean_window_constraints,
    add_capacity_constraints,
    add_initial_rotation_constraints,
    add_irrigated_regime_constraints,
    add_irrigated_second_season_choice,
    add_rotation_constraints,
    sales_groups,
)
from q2_config import DEFAULT_Q2_CONFIG, Q2Config
from q2_uncertainty import Q2ScenarioPaths, cost_multiplier, lower_tail_cvar

ScenarioSalesKey = tuple[int, SalesKey]

class Q2ModelError(RuntimeError):
    """Raised when a Q2 model violates its contract or fails to solve."""

@dataclass(frozen=True)
class Q2VariableBlock:
    x: dict[PlantKey, int]
    z: dict[PlantKey, int]
    regime: dict[RegimeKey, int]
    normal_sales: dict[ScenarioSalesKey, int]
    scenario_profit: dict[int, int]
    tau: int | None
    shortfall: dict[int, int]
    names: tuple[str, ...]
    lower_bounds: np.ndarray
    upper_bounds: np.ndarray

```

```

    integrality: np.ndarray
    objective: np.ndarray

@dataclass(frozen=True)
class Q2BuiltModel:
    data: Q1Data
    scenarios: Q2ScenarioPaths
    config: Q2Config
    objective_kind: str
    cvar_floor_yuan: float | None
    expected_profit_floor_yuan: float | None
    variables: Q2VariableBlock
    constraints: object

@dataclass(frozen=True)
class Q2Solution:
    model: Q2BuiltModel
    result: OptimizeResult
    values: np.ndarray
    solve_seconds: float

    @property
    def x(self) -> dict[PlantKey, float]:
        return {
            key: 0.0 if abs(self.values[index]) <= NUMERICAL_ZERO else float(self.values[index])
            for key, index in self.model.variables.x.items()
        }

    @property
    def z(self) -> dict[PlantKey, float]:
        return {
            key: float(self.values[index])
            for key, index in self.model.variables.z.items()
        }

    @property
    def regimes(self) -> dict[RegimeKey, float]:
        return {
            key: float(self.values[index])
            for key, index in self.model.variables.regime.items()
        }

    @property
    def normal_sales(self) -> dict[ScenarioSalesKey, float]:
        return {
            key: 0.0 if abs(self.values[index]) <= NUMERICAL_ZERO else float(self.values[index])
            for key, index in self.model.variables.normal_sales.items()
        }

    @property
    def scenario_profits(self) -> np.ndarray:
        return np.asarray(
            [
                self.values[self.model.variables.scenario_profit[omega]]
                for omega in self.model.scenarios.scenario_ids
            ],
            dtype=float,
        )

    @property
    def expected_profit_yuan(self) -> float:
        return float(np.dot(self.model.scenarios.proBABILITIES, self.scenario_profits))

    @property
    def empirical_cvar_yuan(self) -> float:
        return lower_tail_cvar(
            self.scenario_profits,
            beta=self.model.config.beta,
            probabilities=self.model.scenarios.proBABILITIES,
        )

    @property
    def auxiliary_cvar_yuan(self) -> float | None:
        variables = self.model.variables
        if variables.tau is None:
            return None
        tau = float(self.values[variables.tau])
        penalty = math.fsum(
            float(self.model.scenarios.proBABILITIES[omega])
            * float(self.values[variables.shortfall[omega]])
            for omega in self.model.scenarios.scenario_ids
        ) / (1.0 - self.model.config.beta)
        return tau - penalty

```

```

def _build_variables(
    data: Q1Data,
    scenarios: Q2ScenarioPaths,
    objective_kind: str,
    config: Q2Config,
) -> Q2VariableBlock:
    valid_objectives = {"risk_neutral", "max_cvar", "expected_with_cvar_floor"}
    if objective_kind not in valid_objectives:
        raise Q2ModelError(f"Unsupported objective kind: {objective_kind}")

    registry = VariableRegistry()
    x: dict[PlantKey, int] = {}
    z: dict[PlantKey, int] = {}
    regime: dict[RegimeKey, int] = {}
    normal_sales: dict[ScenarioSalesKey, int] = {}
    scenario_profit: dict[int, int] = {}
    shortfall: dict[int, int] = {}

    for key in data.compatible_cells:
        plot, crop_id, year, season = key
        area = data.plots[plot].area_mu
        x[key] = registry.add(f"x[{plot}],{crop_id},{year},{season}]", upper=area)
        z[key] = registry.add(
            f"z[{plot}],{crop_id},{year},{season}]", upper=1.0, integer=True
        )

    for plot in data.irrigated_plots:
        for year in YEARS:
            regime[(plot, year)] = registry.add(
                f"u_rice[{plot},{year}]", upper=1.0, integer=True
            )

    groups = sales_groups(data)
    for omega in scenarios.scenario_ids:
        for key in groups:
            crop_id, land_type, year, season = key
            normal_sales[(omega, key)] = registry.add(
                f"normal[{omega}],{crop_id},{land_type},{year},{season}]"
            )
            scenario_profit[omega] = registry.add(
                f"profit[{omega}]",
                lower=-math.inf,
                upper=math.inf,
                objective=(
                    -float(scenarios.proBABILITIES[omega])
                    if objective_kind in {"risk_neutral", "expected_with_cvar_floor"}
                    else 0.0
                ),
            )

    tau: int | None = None
    if objective_kind in {"max_cvar", "expected_with_cvar_floor"}:
        tau = registry.add(
            "cvar_tau",
            lower=-math.inf,
            upper=math.inf,
            objective=-1.0 if objective_kind == "max_cvar" else 0.0,
        )

    for omega in scenarios.scenario_ids:
        shortfall[omega] = registry.add(
            f"cvar_shortfall[{omega}]",
            objective=(
                float(scenarios.proBABILITIES[omega])
                / (1.0 - config.beta)
                if objective_kind == "max_cvar"
                else 0.0
            ),
        )

    return Q2VariableBlock(
        x=x,
        z=z,
        regime=regime,
        normal_sales=normal_sales,
        scenario_profit=scenario_profit,
        tau=tau,
        shortfall=shortfall,
        names=tuple(registry.names),
        lower_bounds=np.asarray(registry.lower, dtype=float),
        upper_bounds=np.asarray(registry.upper, dtype=float),
        integrality=np.asarray(registry.integrality, dtype=np.uint8),
        objective=np.asarray(registry.objective, dtype=float),
    )

def _add_agricultural_constraints(

```

```

    builder: SparseConstraintBuilder, data: Q1Data, variables: Q2VariableBlock
) -> None:
    add_capacity_constraints(builder, data, variables)
    add_activation_constraints(builder, data, variables)
    add_irrigated_regime_constraints(builder, data, variables)
    add_irrigated_second_season_choice(builder, data, variables)
    add_rotation_constraints(builder, data, variables)
    add_initial_rotation_constraints(builder, data, variables)
    add_bean_window_constraints(builder, data, variables)

def _add_scenario_economic_constraints(
    builder: SparseConstraintBuilder,
    data: Q1Data,
    scenarios: Q2ScenarioPaths,
    variables: Q2VariableBlock,
    config: Q2Config,
) -> None:
    groups = sales_groups(data)
    x_by_group: dict[SalesKey, list[tuple[PlantKey, int]]] = {key: [] for key in groups}
    for key, index in variables.x.items():
        plot, crop_id, year, season = key
        group = (crop_id, data.plots[plot].land_type, year, season)
        x_by_group[group].append((key, index))

    demand_groups: dict[tuple[int, int, str], list[SalesKey]] = {}
    for group in groups:
        crop_id, _land_type, year, season = group
        demand_groups.setdefault((crop_id, year, season), []).append(group)

    for omega in scenarios.scenario_ids:
        profit_coefficients: list[tuple[int, float]] = [
            (variables.scenario_profit[omega], 1.0)
        ]
        for group in groups:
            crop_id, land_type, year, season = group
            parameter = data.parameters[(crop_id, land_type, season)]
            yield_value = parameter.yield_jin_per_mu * scenarios.yield_multiplier[
                (omega, crop_id, year)
            ]
            price = parameter.midpoint_price_yuan_per_jin * scenarios.price_multiplier[
                (omega, crop_id, year)
            ]
            normal_index = variables.normal_sales[(omega, group)]
            production_coefficients = [
                (index, -yield_value) for _key, index in x_by_group[group]
            ]
            builder.add(
                [(normal_index, 1.0)] + production_coefficients,
                upper=0.0,
                category="scenario_normal_le_production",
                name=(
                    f"normal_le_production[{omega},{crop_id},{land_type},{year},{season}]"
                ),
            )
            profit_coefficients.append(
                (normal_index, -(1.0 - config.surplus_discount) * price)
            )
        for plant_key, index in x_by_group[group]:
            _plot, _crop, _year, _season = plant_key
            cost = parameter.cost_yuan_per_mu * cost_multiplier(year, config)
            profit_coefficients.append(
                (index, cost - config.surplus_discount * price * yield_value)
            )

    for (crop_id, year, season), demand_sales_groups in demand_groups.items():
        demand = scenarios.demand_jin[(omega, crop_id, year, season)]
        builder.add(
            (
                (variables.normal_sales[(omega, group)], 1.0)
                for group in demand_sales_groups
            ),
            upper=demand,
            category="scenario_demand",
            name=f"demand[{omega},{crop_id},{year},{season}]",
        )

    builder.add(
        profit_coefficients,
        lower=0.0,
        upper=0.0,
        category="scenario_profit",
        name=f"profit_definition[{omega}]",
    )

```

```

def _add_cvar_constraints(
    builder: SparseConstraintBuilder,
    scenarios: Q2ScenarioPaths,
    variables: Q2VariableBlock,
    config: Q2Config,
    cvar_floor_yuan: float | None,
) -> None:
    if variables.tau is None:
        if cvar_floor_yuan is not None:
            raise Q2ModelError("CVaR floor requires CVaR auxiliary variables")
        return
    for omega in scenarios.scenario_ids:
        builder.add(
            {
                variables.tau: 1.0,
                variables.scenario_profit[omega]: -1.0,
                variables.shortfall[omega]: -1.0,
            },
            upper=0.0,
            category="cvar_shortfall",
            name=f"cvar_shortfall[{omega}]",
        )
    if cvar_floor_yuan is not None:
        coefficients: list[tuple[int, float]] = [(variables.tau, 1.0)]
        coefficients.extend(
            (
                variables.shortfall[omega],
                -float(scenarios.proBABILITIES[omega]) / (1.0 - config.beta),
            )
            for omega in scenarios.scenario_ids
        )
        builder.add(
            coefficients,
            lower=float(cvar_floor_yuan),
            category="cvar_floor",
            name=f"cvar_floor[{cvar_floor_yuan:.12g}]",
        )

def _add_expected_profit_floor(
    builder: SparseConstraintBuilder,
    scenarios: Q2ScenarioPaths,
    variables: Q2VariableBlock,
    expected_profit_floor_yuan: float | None,
) -> None:
    if expected_profit_floor_yuan is None:
        return
    builder.add(
        (
            (variables.scenario_profit[omega], float(scenarios.proBABILITIES[omega]))
            for omega in scenarios.scenario_ids
        ),
        lower=float(expected_profit_floor_yuan),
        category="expected_profit_floor",
        name=f"expected_profit_floor[{expected_profit_floor_yuan:.12g}]",
    )

def build_q2_model(
    data: Q1Data,
    scenarios: Q2ScenarioPaths,
    *,
    objective_kind: str,
    cvar_floor_yuan: float | None = None,
    expected_profit_floor_yuan: float | None = None,
    config: Q2Config = DEFAULT_Q2_CONFIG,
) -> Q2BuiltModel:
    if not math.isclose(
        math.fsum(float(value) for value in scenarios.proBABILITIES),
        1.0,
        rel_tol=0.0,
        abs_tol=1e-12,
    ):
        raise Q2ModelError("Scenario probabilities must sum to one")
    if objective_kind == "expected_with_cvar_floor" and cvar_floor_yuan is None:
        raise Q2ModelError("Expected-profit frontier model requires a CVaR floor")
    if objective_kind == "risk_neutral" and cvar_floor_yuan is not None:
        raise Q2ModelError("CVaR floor requires a model with CVaR auxiliaries")
    if cvar_floor_yuan is not None and not math.isfinite(cvar_floor_yuan):
        raise Q2ModelError("CVaR floor must be finite")
    if expected_profit_floor_yuan is not None and not math.isfinite(
        expected_profit_floor_yuan
    ):
        raise Q2ModelError("Expected-profit floor must be finite")

    variables = _build_variables(data, scenarios, objective_kind, config)

```

```

builder = SparseConstraintBuilder(len(variables.names))
_add_agricultural_constraints(builder, data, variables)
_add_scenario_economic_constraints(builder, data, scenarios, variables, config)
_add_cvar_constraints(builder, scenarios, variables, config, cvar_floor_yuan)
_add_expected_profit_floor(
    builder, scenarios, variables, expected_profit_floor_yuan
)
constraints = builder.build()
return Q2BuiltModel(
    data=data,
    scenarios=scenarios,
    config=config,
    objective_kind=objective_kind,
    cvar_floor_yuan=cvar_floor_yuan,
    expected_profit_floor_yuan=expected_profit_floor_yuan,
    variables=variables,
    constraints=constraints,
)

def build_lexicographic_stage2_model(
    data: Q1Data,
    scenarios: Q2ScenarioPaths,
    *,
    incumbent_cvar_yuan: float,
    expected_profit_floor_yuan: float | None = None,
    config: Q2Config = DEFAULT_Q2_CONFIG,
) -> Q2BuiltModel:
    """Build the approved second-stage endpoint without an arbitrary lambda.

    Stage 1 supplies the feasible incumbent lower-tail profit CVaR. Stage 2
    preserves at least that achieved level and maximizes expected profit.
    """

    if not math.isfinite(incumbent_cvar_yuan):
        raise Q2ModelError("Stage-1 incumbent CVaR must be finite")
    return build_q2_model(
        data,
        scenarios,
        objective_kind="expected_with_cvar_floor",
        cvar_floor_yuan=float(incumbent_cvar_yuan),
        expected_profit_floor_yuan=expected_profit_floor_yuan,
        config=config,
    )

def build_q2_lp_relaxation(model: Q2BuiltModel) -> Q2BuiltModel:
    """Return the exact model with only integer restrictions relaxed.

    Every variable retains its original finite/infinite bounds. Objective
    coefficients, variable indices, scenarios, and the full sparse constraint
    system are unchanged. Copies prevent the relaxation from mutating the
    source MILP's arrays.
    """

    if model.objective_kind != "max_cvar":
        raise Q2ModelError("LP upper-bound relaxation requires a max_cvar model")
    variables = replace(
        model.variables,
        lower_bounds=model.variables.lower_bounds.copy(),
        upper_bounds=model.variables.upper_bounds.copy(),
        integrality=np.zeros_like(model.variables.integrality),
        objective=model.variables.objective.copy(),
    )
    return replace(model, variables=variables)

def solve_q2_model(
    model: Q2BuiltModel,
    *,
    time_limit_seconds: float,
    mip_relative_gap: float = 0.005,
    display_solver_log: bool = False,
) -> Q2Solution:
    import time

    started = time.perf_counter()
    result = milp(
        c=model.variables.objective,
        integrality=model.variables.integrality,
        bounds=Bounds(model.variables.lower_bounds, model.variables.upper_bounds),
        constraints=model.constraints.linear_constraint,
        options={
            "disp": display_solver_log,
            "presolve": True,
            "time_limit": float(time_limit_seconds),

```

```

        "mip_rel_gap": float(mip_relative_gap),
    },
)
solve_seconds = time.perf_counter() - started
if result.status != 0 or result.x is None or result.fun is None:
    raise Q2ModelError(
        f"{model.objective_kind} failed: status={result.status}, "
        f"success={result.success}, message={result.message}"
    )
values = np.asarray(result.x, dtype=float)
if not np.all(np.isfinite(values)):
    raise Q2ModelError("Solver returned non-finite variable values")
return Q2Solution(model=model, result=result, values=values, solve_seconds=solve_seconds)

```

B.4 问题三：共同因子、稀疏弹性与相关优化

Listing 4: source/src/q3_scenarios.py

```

"""Bounded Q3 correlated scenarios with sparse price-demand interactions."""

from __future__ import annotations

import math
from dataclasses import dataclass

import numpy as np
from scipy.stats import norm

from q1_data import Q1Data, YEARS
from q2_config import UniformSupport
from q3_config import (
    BASELINE_Q3_CONFIG,
    CROP_GROUPS,
    CROP_TO_GROUP,
    NONSTARCHY_VEGETABLE_GROUPS,
    Q3Config,
)

InnovationKey = tuple[str, int, int]
DemandKey = tuple[int, int, int, str]
MultiplierKey = tuple[int, int, int]
InteractionKey = tuple[int, int, int]

@dataclass(frozen=True)
class Q3ScenarioPaths:
    count: int
    seed: int
    sampling_method: str
    probabilities: np.ndarray
    dimension_order: tuple[InnovationKey, ...]
    unit_draws: np.ndarray
    innovations: dict[InnovationKey, np.ndarray]
    supports: dict[InnovationKey, UniformSupport]
    factor_draws: dict[tuple[str, str | int, int], np.ndarray]
    base_demand_jin: dict[DemandKey, float]
    demand_jin: dict[DemandKey, float]
    demand_interaction_multiplier: dict[InteractionKey, float]
    yield_multiplier: dict[MultiplierKey, float]
    price_multiplier: dict[MultiplierKey, float]
    cost_multiplier: dict[MultiplierKey, float]
    config: Q3Config

    @property
    def scenario_ids(self) -> tuple[int, ...]:
        return tuple(range(self.count))

def _scale(values: np.ndarray, support: UniformSupport) -> np.ndarray:
    return support.lower + (support.upper - support.lower) * values

def _lhs_marginal_from_latent(
    latent: np.ndarray, rng: np.random.Generator
) -> np.ndarray:
    count = latent.size
    marginal = (rng.permutation(count) + rng.random(count)) / count
    ordered_values = np.sort(marginal)
    output = np.empty(count, dtype=float)
    output[np.argsort(latent, kind="stable")] = ordered_values
    return output

```

```

def _unit_from_latent(
    latent: np.ndarray, method: str, rng: np.random.Generator
) -> np.ndarray:
    if method == "lhs":
        return _lhs_marginal_from_latent(latent, rng)
    if method != "mc":
        raise ValueError(f"Unsupported Q3 sampling method: {method}")
    return np.clip(norm.cdf(latent), np.finfo(float).eps, 1.0 - np.finfo(float).eps)

def _support_for(data: Q1Data, family: str, crop_id: int, config: Q3Config) -> UniformSupport:
    if family == "demand_growth":
        return config.wheat_corn_growth
    if family == "demand_delta":
        return config.other_demand_delta
    if family == "yield_delta":
        return config.yield_delta
    if family == "cost_growth":
        return config.cost_growth
    if family == "price_change":
        crop = data.crops[crop_id]
        if crop.is_grain:
            return config.grain_price_growth
        if crop.is_vegetable:
            return config.vegetable_price_growth
        if crop_id in {38, 39, 40}:
            return config.mushroom_decline
        if crop_id == 41:
            return config.morel_decline
    raise ValueError(f"Unsupported Q3 support family={family}, crop={crop_id}")

def _q2_reference_price_multiplier(data: Q1Data, crop_id: int, year: int) -> float:
    periods = year - 2023
    crop = data.crops[crop_id]
    if crop.is_grain:
        return 1.0
    if crop.is_vegetable:
        return 1.05**periods
    if crop_id in {38, 39, 40}:
        return 0.97**periods
    if crop_id == 41:
        return 0.95**periods
    raise ValueError(crop_id)

def _complement_crop_ids(crop_id: int) -> tuple[int, ...]:
    group = CROP_TO_GROUP[crop_id]
    if group == "staple_grains":
        return tuple(
            crop for name in NONSTARCHY_VEGETABLE_GROUPS for crop in CROP_GROUPS[name]
        )
    if group in NONSTARCHY_VEGETABLE_GROUPS:
        return CROP_GROUPS["staple_grains"]
    return ()

def generate_q3_scenarios(
    data: Q1Data,
    *,
    count: int,
    seed: int,
    sampling_method: str,
    config: Q3Config = BASELINE_Q3_CONFIG,
) -> Q3ScenarioPaths:
    if count <= 0:
        raise ValueError("Q3 scenario count must be positive")
    rng = np.random.default_rng(seed)
    factor_draws: dict[tuple[str, str | int, int], np.ndarray] = {}
    for year in YEARS:
        factor_draws[("weather", 0, year)] = rng.standard_normal(count)
        factor_draws[("input", 0, year)] = rng.standard_normal(count)
        for group in CROP_GROUPS:
            factor_draws[("market", group, year)] = rng.standard_normal(count)

    dimension_order: list[InnovationKey] = []
    units: list[np.ndarray] = []
    innovations: dict[InnovationKey, np.ndarray] = {}
    supports: dict[InnovationKey, UniformSupport] = {}
    for crop_id in sorted(data.crops):
        group = CROP_TO_GROUP[crop_id]
        for year in YEARS:
            idiosyncratic = {
                family: rng.standard_normal(count)
                for family in ("demand", "yield", "price", "cost")

```

```

    }
    demand_family = "demand_growth" if crop_id in {6, 7} else "demand_delta"
    latent_by_family = {
        demand_family: (
            config.demand_market_loading * factor_draws[("market", group, year)]
            + math.sqrt(1.0 - config.demand_market_loading**2)
            * idiosyncratic["demand"]
        ),
        "yield_delta": (
            config.yield_weather_loading * factor_draws[("weather", 0, year)]
            + math.sqrt(1.0 - config.yield_weather_loading**2)
            * idiosyncratic["yield"]
        ),
        "price_change": (
            config.price_market_loading * factor_draws[("market", group, year)]
            + config.price_input_loading * factor_draws[("input", 0, year)]
            + math.sqrt(
                1.0
                - config.price_market_loading**2
                - config.price_input_loading**2
            )
            * idiosyncratic["price"]
        ),
        "cost_growth": (
            config.cost_input_loading * factor_draws[("input", 0, year)]
            + math.sqrt(1.0 - config.cost_input_loading**2)
            * idiosyncratic["cost"]
        ),
    }
    for family, latent in latent_by_family.items():
        key = (family, crop_id, year)
        unit = _unit_from_latent(latent, sampling_method, rng)
        support = _support_for(data, family, crop_id, config)
        dimension_order.append(key)
        units.append(unit)
        supports[key] = support
        innovations[key] = _scale(unit, support)

unit_draws = np.column_stack(units)
probabilities = np.full(count, 1.0 / count, dtype=float)
yield_multiplier: dict[MultiplierKey, float] = {}
price_multiplier: dict[MultiplierKey, float] = {}
cost_multiplier: dict[MultiplierKey, float] = {}
for omega in range(count):
    for crop_id in sorted(data.crops):
        price_level = 1.0
        cost_level = 1.0
        for year in YEARS:
            yield_multiplier[(omega, crop_id, year)] = 1.0 + innovations[
                ("yield_delta", crop_id, year)
            ][omega]
            price_change = innovations[("price_change", crop_id, year)][omega]
            if crop_id >= 38:
                price_level *= 1.0 - price_change
            else:
                price_level *= 1.0 + price_change
            price_multiplier[(omega, crop_id, year)] = price_level
            cost_level *= 1.0 + innovations[("cost_growth", crop_id, year)][omega]
            cost_multiplier[(omega, crop_id, year)] = cost_level

sales_crop_seasons = sorted(
    {(crop_id, season) for plot, crop_id, _year, season in data.compatible_cells}
)
base_demand: dict[DemandKey, float] = {}
demand: dict[DemandKey, float] = {}
interaction_multiplier: dict[InteractionKey, float] = {}
lower_log = math.log(config.demand_interaction_lower)
upper_log = math.log(config.demand_interaction_upper)
for omega in range(count):
    wheat_corn_level = {6: 1.0, 7: 1.0}
    for year in YEARS:
        relative_price = {
            crop_id: price_multiplier[(omega, crop_id, year)]
            / _q2_reference_price_multiplier(data, crop_id, year)
            for crop_id in data.crops
        }
    for crop_id in sorted(data.crops):
        if crop_id in {6, 7}:
            wheat_corn_level[crop_id] *= 1.0 + innovations[
                ("demand_growth", crop_id, year)
            ][omega]
            base_multiplier = wheat_corn_level[crop_id]
        else:
            base_multiplier = 1.0 + innovations[
                ("demand_delta", crop_id, year)
            ][omega]

```

```

        group_peers = tuple(
            peer for peer in CROP_GROUPS[CROP_TO_GROUP[crop_id]] if peer != crop_id
        )
        peer_log = (
            math.fsum(math.log(relative_price[peer]) for peer in group_peers)
            / len(group_peers)
            if group_peers
            else 0.0
        )
        complement_ids = _complement_crop_ids(crop_id)
        complement_log = (
            math.fsum(math.log(relative_price[peer]) for peer in complement_ids)
            / len(complement_ids)
            if complement_ids
            else 0.0
        )
        scale = config.elasticity_scale
        log_adjustment = scale * (
            config.own_price_elasticity * math.log(relative_price[crop_id])
            + config.substitute_elasticity * peer_log
            + config.complement_elasticity * complement_log
        )
        adjustment = math.exp(min(upper_log, max(lower_log, log_adjustment)))
        interaction_multiplier[(omega, crop_id, year)] = adjustment
        for demand_crop_id, season in sales_crop_seasons:
            if demand_crop_id != crop_id:
                continue
            baseline = data.expected_sales_proxy.get((crop_id, season), 0.0)
            key = (omega, crop_id, year, season)
            base_demand[key] = baseline * base_multiplier
            demand[key] = base_demand[key] * adjustment

    return Q3ScenarioPaths(
        count=count,
        seed=seed,
        sampling_method=sampling_method,
        probabilities=probabilities,
        dimension_order=tuple(dimension_order),
        unit_draws=unit_draws,
        innovations=innovations,
        supports=supports,
        factor_draws=factor_draws,
        base_demand_jin=base_demand,
        demand_jin=demand,
        demand_interaction_multiplier=interaction_multiplier,
        yield_multiplier=yield_multiplier,
        price_multiplier=price_multiplier,
        cost_multiplier=cost_multiplier,
        config=config,
    )

```

Listing 5: source/src/q3_model.py

```

"""Q3 stochastic model with scenario-specific dependent economics."""

from __future__ import annotations

import math

from q1_data import Q1Data
from q1_model import PlantKey, SalesKey, SparseConstraintBuilder, sales_groups
from q2_config import Q2Config
from q2_model import (
    Q2BuiltModel,
    Q2ModelError,
    Q2Solution,
    _add_agricultural_constraints,
    _add_cvar_constraints,
    _add_expected_profit_floor,
    _build_variables,
    build_q2_lp_relaxation,
    solve_q2_model,
)
from q3_config import BASELINE_Q3_CONFIG, Q3Config
from q3_scenarios import Q3ScenarioPaths

def _objective_config(config: Q3Config) -> Q2Config:
    return Q2Config(beta=config.beta, surplus_discount=config.surplus_discount)

def _add_q3_economic_constraints(
    builder: SparseConstraintBuilder,
    data: Q1Data,
    scenarios: Q3ScenarioPaths,

```

```

variables,
config: Q3Config,
) -> None:
    groups = sales_groups(data)
    x_by_group: dict[SalesKey, list[tuple[PlantKey, int]]] = {key: [] for key in groups}
    for key, index in variables.x.items():
        plot, crop_id, year, season = key
        group = (crop_id, data.plots[plot].land_type, year, season)
        x_by_group[group].append((key, index))
    demand_groups: dict[tuple[int, int, str], list[SalesKey]] = {}
    for group in groups:
        crop_id, _land_type, year, season = group
        demand_groups.setdefault((crop_id, year, season), []).append(group)

    for omega in scenarios.scenario_ids:
        profit_coefficients: list[tuple[int, float]] = [
            (variables.scenario_profit[omega], 1.0)
        ]
        for group in groups:
            crop_id, land_type, year, season = group
            parameter = data.parameters[(crop_id, land_type, season)]
            yield_value = parameter.yield_jin_per_mu * scenarios.yield_multiplier[
                (omega, crop_id, year)
            ]
            price = parameter.midpoint_price_yuan_per_jin * scenarios.price_multiplier[
                (omega, crop_id, year)
            ]
            normal_index = variables.normal_sales[(omega, group)]
            builder.add(
                [(normal_index, 1.0)]
                + [(index, -yield_value) for _key, index in x_by_group[group]],
                upper=0.0,
                category="scenario_normal_le_production",
                name=f"normal_le_production[{omega},{crop_id},{land_type},{year},{season}]",
            )
            profit_coefficients.append(
                (normal_index, -(1.0 - config.surplus_discount) * price)
            )
            for _plant_key, index in x_by_group[group]:
                cost = parameter.cost_yuan_per_mu * scenarios.cost_multiplier[
                    (omega, crop_id, year)
                ]
                profit_coefficients.append(
                    (index, cost - config.surplus_discount * price * yield_value)
                )

            for (crop_id, year, season), demand_sales_groups in demand_groups.items():
                builder.add(
                    (
                        (variables.normal_sales[(omega, group)], 1.0)
                        for group in demand_sales_groups
                    ),
                    upper=scenarios.demand_jin[(omega, crop_id, year, season)],
                    category="scenario_demand",
                    name=f"demand[{omega},{crop_id},{year},{season}]",
                )
                builder.add(
                    profit_coefficients,
                    lower=0.0,
                    upper=0.0,
                    category="scenario_profit",
                    name=f"profit_definition[{omega}]",
                )

def build_q3_model(
    data: Q1Data,
    scenarios: Q3ScenarioPaths,
    *,
    objective_kind: str,
    cvar_floor_yuan: float | None = None,
    expected_profit_floor_yuan: float | None = None,
    config: Q3Config | None = None,
) -> Q2BuiltModel:
    config = scenarios.config if config is None else config
    if config != scenarios.config:
        raise Q2ModelError("Q3 model config must match the materialized scenarios")
    if not math.isclose(
        math.fsum(float(value) for value in scenarios.probabilities),
        1.0,
        rel_tol=0.0,
        abs_tol=1e-12,
    ):
        raise Q2ModelError("Q3 scenario probabilities must sum to one")
    objective_config = _objective_config(config)
    variables = _build_variables(data, scenarios, objective_kind, objective_config)

```

```

builder = SparseConstraintBuilder(len(variables.names))
_add_agricultural_constraints(builder, data, variables)
_add_q3_economic_constraints(builder, data, scenarios, variables, config)
_add_cvar_constraints(
    builder, scenarios, variables, objective_config, cvar_floor_yuan
)
_add_expected_profit_floor(
    builder, scenarios, variables, expected_profit_floor_yuan
)
return Q2BuiltModel(
    data=data,
    scenarios=scenarios,
    config=objective_config,
    objective_kind=objective_kind,
    cvar_floor_yuan=cvar_floor_yuan,
    expected_profit_floor_yuan=expected_profit_floor_yuan,
    variables=variables,
    constraints=builder.build(),
)

def build_q3_lp_relaxation(model: Q2BuiltModel) -> Q2BuiltModel:
    return build_q2_lp_relaxation(model)

def solve_q3_model(
    model: Q2BuiltModel,
    *,
    time_limit_seconds: float,
    mip_relative_gap: float = 0.005,
    display_solver_log: bool = False,
) -> Q2Solution:
    return solve_q2_model(
        model,
        time_limit_seconds=time_limit_seconds,
        mip_relative_gap=mip_relative_gap,
        display_solver_log=display_solver_log,
    )

```